Designing Production Agentic Systems
Lecture 3: Execution machinery and vulnerability triage

Rahul Krishna
Columbia University, COMS E6998-019, Fall 2026

Revised October 16, 2026, as reading for the midterm and the final, from the lecture of September 25, 2026

Abstract

These notes are the reading for Lecture 3, written for students preparing for the midterm and the final. The first three sections describe how an agent executes, using the planning agent of Lecture 2, the version of `mini-swe-agent` with a planner and a validator: we describe it as an extended state machine whose runs follow paths through a computation tree, show how the runtime stores its state as a record of fields that reducers merge, and show how that record routes a run, sends it back along feedback edges and ends it. They also state what a call budget, a stall check and a trace can and cannot establish about how a run ended. The last two sections introduce the problem of the first homework: vulnerability management, the discovery of alerts with the static-analysis scanner Bandit, and the triage that decides whether an attacker can reach the flaw behind each alert. Each numbered section ends with a list of key points for revision.

Contents

1 Agents as state machines **1**
1.1 A state machine as a description of an agent 1
1.2 Control locations of the planning agent 2
1.3 State, inputs and outputs 2
1.4 Extended state machines and configurations 2
1.5 Transitions: guards, output actions and set actions 3
1.6 Behaviors, computation trees and traces 4
1.7 The planning agent as an extended state machine . 4
1.8 A run as a trajectory 5
1.9 What the trace records and what we infer 6

2 State, records and reducers **6**
2.1 State records and schemas 6
2.2 Partial updates 7
2.3 The controller's return sites 7
2.4 Model context, runtime state and durable state . . 8
2.5 Reducers 9
2.6 Field lifetimes 9
2.7 The call budget as an invariant 10
2.8 Building and compiling the graph 11
2.9 Designing a state graph 11

3 Routing, feedback and stopping **11**
3.1 Routing functions and conditional edges 11
3.2 The same graph, different paths 12
3.3 Feedback edges and their bounds 12
3.4 Loop bounds and termination 13
3.5 What a trace establishes 14
3.6 A recorded repair and a keyboard interruption . . . 15

4 Vulnerability management and discovery (the HW1 domain) **15**
4.1 Vulnerability management and its stages 16
4.2 Kinds of discovery tool 16
4.3 Rules and what they match 16
4.4 What an alert contains 17
4.5 The attack surface 18
4.6 The threat model 18
4.7 The OWASP Top 10 18
4.8 Weaknesses and their catalogues 19
4.9 From a family to a line of Python 19

5 Bandit alerts and triage **20**
5.1 How Bandit reads a program 20
5.2 A rule in Bandit's source 20
5.3 Severity and confidence 21
5.4 One run and one alert record 21
5.5 Vulnerability and exploit 22
5.6 SQL injection: a real alert and a false positive . . . 22
5.7 Code injection: `exec` and `eval` 23
5.8 A weak hash 23
5.9 An XML alert and an unreported XPath query . . . 23
5.10 What discovery leaves undecided 24
5.11 The cost of each kind of mistake 24
5.12 Prioritization 24
5.13 What a label must carry 25

1 Agents as state machines

Lecture 2 drew `mini-swe-agent` as a state machine, and its Section 4 built a version of the agent with a planner and a validator as a graph. This section makes that drawing precise by describing the agent as an extended state machine, with control locations, variables and guarded transitions. We then read a run as a path through a tree of possible runs and state what its trace records. These definitions let us say which steps a run can take and why it ended, and which of those facts the trace supports.

Section 2 and Section 3 show how the runtime, the code that runs the agent (Lecture 2), stores the variables, chooses the next step and ends a run, and Lecture 4 covers how it schedules and executes tools. Section 4 and Section 5 introduce the problem of the first homework, HW1: deciding which alerts of a static-analysis scanner point to flaws that an attacker can reach.

Our running example in this section and the next two is the **planning agent** of Lecture 2, Section 4. It extends the reactive loop of `mini-swe-agent` (Lecture 2, Section 3), in which the model chooses each next action after reading the result of the previous one. The planning agent adds two nodes to the loop: a planner, which writes the steps the run intends to take, and a validator, which decides whether to accept a patch. Like the loop, it repairs one scanner alert at a time in OWASP BenchmarkPython, the Python web application of Lecture 2, which is made up of many small test cases.

1.1 A state machine as a description of an agent

A state machine is a simplified description of an agent. It omits the prompts, the model and the tool code, and it keeps only what decides how a run proceeds. An agent runs in discrete steps, such as model calls, tool calls and checks. Between two steps the agent keeps some information, and the program we wrote decides which stored values may change and which computation may run next.

Lecture 2 described a state machine as a set of states with transitions between them, and its states were the agent's modes. In this lecture a state is the information the agent keeps between steps, and Section 1.2 gives the modes their own name. A complete description of a state machine has the following parts [25, slide 17]:

- **States.** The states are the values that the information kept between steps can take.
- **Inputs.** The inputs are the results the agent can receive during a step.
- **Outputs.** The outputs are what a step can emit, such as a request to run a tool.

- **An initial configuration.** A configuration is the point a run has reached together with the values of the information it keeps, as Section 1.4 defines precisely. The initial configuration fixes where every run begins and the values it begins with.
- **A transition rule.** The transition rule specifies, for a state and an input, how the state changes and what output the step produces.

A workflow that is meant to end also specifies where its runs end. These are the terminal configurations of Section 1.4, from which the runtime schedules no further step.

Because it leaves out the prompts, the model and the tool code, the description lets us examine how the agent executes, which steps can follow which and when a run ends, without describing the whole program. The model itself is not part of the description. Its replies enter the machine only as inputs (Section 1.3).

1.2 Control locations of the planning agent

A **control location** names the computation to run next, or marks the end of the run. In a drawing of the machine, each control location is a node and each arrow is a **transition**, a permitted step from one location to another. Lecture 2 called the Controller, Tool and Submit of `mini-swe-agent` its states. In this lecture they are control locations.

The planning agent has the following control locations (Figure 1):

- `plan`. The planner calls a model to write the plan. It calls the model again to revise the plan after the run stalls, which in Lecture 2 meant that three tool calls in a row returned the same result. Section 1.7 refines this condition.
- `controller`. The controller calls a model whose reply either requests tools or, by requesting none, proposes to submit.
- `tools`. This location runs the requested tools, `read_file`, `search` or `edit`, and records their results.
- `submit`. This location runs three mechanical checks on the candidate patch: that it applies, that it touches the line the alert flagged, and that a rescan of the patched code no longer reports the alert.
- `validate`. The validator calls a model that accepts or rejects the patch.
- `START` and `END`. `START` runs no computation. It marks where a run enters the machine, and the arrow from it gives the variables their starting values. `END` marks the end of a run.

The locations `plan`, `controller` and `validate` call a model, and Figure 1 draws them in blue. The locations `tools` and `submit` run tools or checks, and the figure draws them in grey.

A run can visit the same location many times, and two visits to the controller can lead to different next steps. What differs between the visits is the information the agent has kept, its state.

1.3 State, inputs and outputs

The state of an agent is the information retained between steps that can affect what the agent does next. Both the model and the runtime use that information. The controller's model reads its instructions and the earlier observations, the results of tool calls as the runtime records them (Lecture 2, Section 3), to choose an action. The runtime reads the call count, the number of model calls made so far, to decide whether another model call is allowed.

Definition 1.1 (State)**.** State is the "condition of the system at a particular point in time". It "[e]ncodes everything about the past that influences the system's reaction to current input" [25, slide 10].

For an agent, the part of the past that reaches its next step through the state is what the agent has kept: the messages exchanged with the model, the observations and the counters, among other values. Anything else from the past can reach a later step only as an input.

An **input** is a result that the agent receives during a step: a model reply, a tool result or the result of a check. The stored state alone does not determine an input. A model reply is an input, so the same state can receive a different reply on another visit or in another run.

The files the agent works on lie outside the state in this description. The agent learns about them through its tools and through the checks that `submit` runs. A tool's report about the files enters the machine as an input, and the runtime then stores it as an observation. The result of an edit, for example, records only that the edit was made, and a later read of that file reports the new content.

An **output action** is the part of a transition that emits an output. The output is either a request for an operation, such as running a tool, or the report of a terminal outcome, such as an accepted patch.

The division between state and input separates what the program decides from what it receives. The state determines which transitions are permitted. It does not determine which result arrives. Every transition the agent can take is written in the program we designed, while the model, the tools and the checks supply the inputs that select among them.

1.4 Extended state machines and configurations

An agent has a small number of control locations and a large amount of data, so we describe it as an extended state machine. An **extended state machine** has finitely many control locations and a set of variables that store data. Its transitions may read and write the variables.

Definition 1.2 (Extended state machine)**.** "Extended state machines augment the FSM model with variables that may be read or written." [26, slide 4]

In the quotation, FSM stands for finite-state machine, a state machine with finitely many states.

A **configuration** pairs the current control location with the current value of every variable. It says where the run is and what the run holds. The variables hold the state of Section 1.3. The configuration adds the control location, so one step can change the location, the variables, or both.

For example, in the recorded run of Section 1.8, just before the controller's third model call, the configuration includes the following location and values:

- **Location.** The run is at `controller`.
- `observations`. Two tool results have been stored.
- `model_calls`. Three model calls have completed, one in `plan` and two in `controller`.
- `status`. The status is `running`. The status is the variable that records whether the run is still going and, once it has ended, how it ended (Lecture 2).
- `messages`. The variable holds the conversation so far.
- `plan`. This variable, which shares its name with the location, holds the plan: the list of steps the run intends to take. Lecture 2 called these steps plan steps, or subgoals.

The configuration also holds the values of the other variables, such as the alert and the token usage. In the same run, the controller is visited five times, and each visit has a different configuration because the stored values have changed between visits.

Every run starts from the **initial configuration**. It pairs the first control location, `plan`, with the starting values of the variables: the opening messages, counters at zero, no observations and the

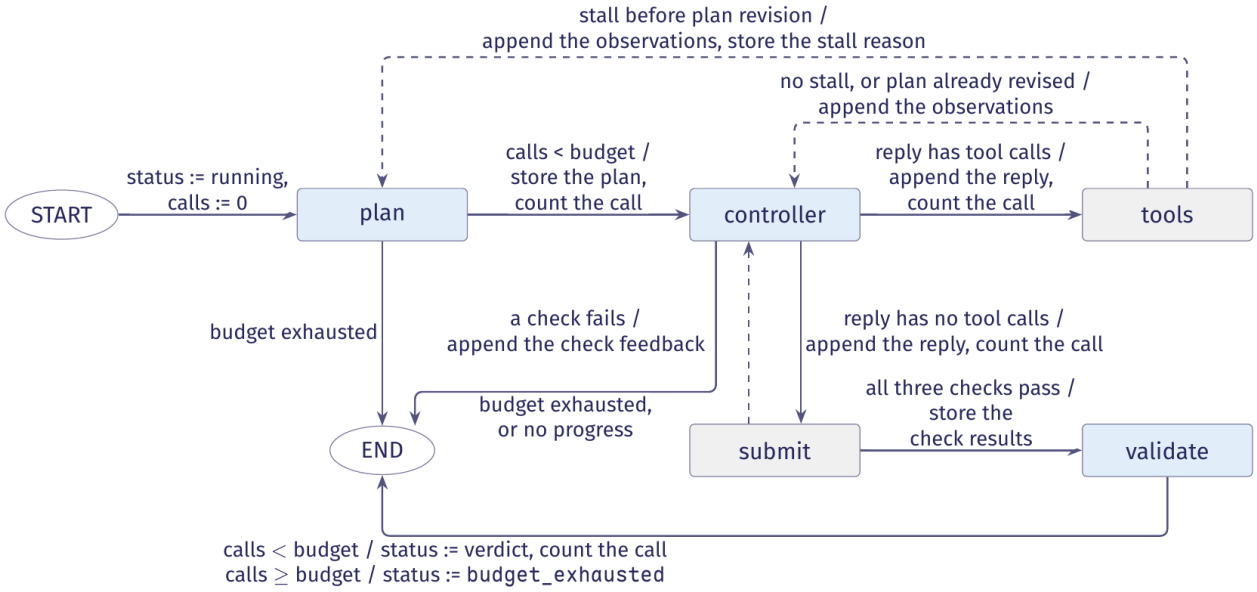


Figure 1: The planning agent as an extended state machine, with the nodes that call a model in blue and the nodes that run tools or checks in grey. Arrows carry guards and, after a slash, set actions, and the dashed arrows return to a location visited earlier.

status `running`. A **terminal configuration** is one from which the runtime schedules no further step, so reaching it ends the run. The state then stores the reason for ending, in the status.

An extended state machine can have finitely many control locations and infinitely many configurations. How many configurations a location has depends on the values its variables can take:

- **A bounded counter.** The call budget, which Lecture 2 called the budget, is the most model calls a run may make. If `model_calls` were the only variable and the call budget were N, the counter could hold the values 0 through N. Each location would then have at most $N+1$ configurations, which is 41 for our budget of 40.
- **An unbounded list.** If a list of messages can grow without a bound on its length, a single control location can have infinitely many configurations.

A configuration is **reachable** when some run can arrive at it from the initial configuration. When a machine has finitely many configurations, an analysis can examine every reachable one, for example to show that no run arrives at an unsafe configuration. When a variable can take infinitely many values, as the unbounded list can, a procedure that tries to list every reachable configuration may never finish. The control locations and the guards (Section 1.5) still form finite sets, and both are written in the program. An argument about the agent can therefore rest on these finite sets and on properties of the variables, without listing configurations. Section 2 and Section 3 argue in this way about the call budget.

1.5 Transitions: guards, output actions and set actions

In an extended state machine, a transition goes from one configuration to the next. Each transition has a guard, an output action and a set action. The **guard** is a predicate, a true-or-false test of the variables and the input, that states when the transition is allowed. The output action emits the step's output (Section 1.3). The **set action** updates the variables.

Extended state machines label each arrow with the guard and the output action, separated by a slash, and with the set action below them [26, slide 5]. We write one transition with its input and the configurations before and after it:

$$(q,v) \xrightarrow{e;\, g\,/\,out;\, set} (q',v')$$

At location q with variable values v, input e arrives. If the guard g holds, the transition emits out, applies set to the variables, and moves to location q' with values v'.

Table 1 gives each element of a transition its symbol and states what it is in an agent. The guard is written $g(v,e)$ because it may test the stored variables, the new input, or both.

Table 1: The elements of a transition in an agent's state machine, with the symbol used in the notation and what each element is in an agent.

Element	Symbol	In an agent
Control location	q	The computation ready to run: a model call, the execution of tools, or a check
Variables	v	Messages, observations, a plan, counters, check results and a status, kept between steps
Input	e	A result received during a step: a model reply, a tool result or a check result
Guard	$g(v,e)$	A true-or-false test of the variables and the input, such as whether the reply requests a tool
Output action	out	A request for an operation, or the report of a terminal outcome
Set action	set	An update of the variables, such as storing a reply, extending the observations or incrementing a counter
Initial configuration	(q_0,v_0)	The first control location with the starting values: opening messages, zero counters, no observations and the status `running`
Terminal configuration	No symbol	A configuration from which no further computation is scheduled, with the reason for ending stored in the state

The transition from the controller to the tools shows each element in code (Listing 1). In the code, one transition is split between two functions. The node `controller` receives the input and returns the set action. The routing function `after_controller` tests the guard. As in Lecture 2, Section 3, a routing function reads the state after a node has finished and returns the name of the next location.

Listing 1: Illustrative code for the transition from the controller to the tools: the node receives the input and returns the set action, and the routing function tests the guard.

```
def controller(state):
    # (the budget and stall checks that run first are omitted)
    reply = call_model(state["messages"])  # the input e
    return {"messages": [reply],          # set: append the reply
            "model_calls": 1,            # set: add one call
            "usage": usage_of(reply)}     # set: add the usage

def after_controller(state):
    if state["status"] != RUNNING:
        return END
    if state["messages"][-1].tool_calls:   # the guard g
        return "tools"
    return "submit"
```

The elements of this transition are the following:

- **Input.** The model's reply is the input e. It arrives from the model during the step.
- **Set action.** The node returns the reply, the number 1 and the reply's token usage. The runtime appends the reply to the stored messages, adds 1 to the stored `model_calls`, and adds the usage to a running total. How the runtime merges a returned value with a stored one is the subject of Section 2.
- **Guard.** The test `state["messages"][-1].tool_calls` is the guard. In the code it runs after the set action, and it reads the reply from its stored copy, the last message. The order makes no difference here, because both transitions that leave the controller after a model call apply the same set action.
- **Output action.** The transition requests that the tools be executed.

The first test in `after_controller` handles a run that the runtime has already stopped. If the status is no longer `running`, the function returns `END` (Section 1.7).

1.6 Behaviors, computation trees and traces

A **behavior** is the sequence of steps that a run takes, leaving out stuttering steps. A **stuttering step** is a step with no input, no change of state and no output. Such a step changes nothing that can be observed, so leaving it out loses no information. Two descriptions of a run that differ only in stuttering steps describe the same behavior.

A **computation tree** represents every possible behavior from the initial configuration. Its root is the initial configuration. Each tree node is the configuration on one visit to a control location, and the children of a tree node are the configurations that one step can reach from it. A run follows one path from the root.

A machine is **deterministic** when, in every configuration, each input enables exactly one transition. A machine is **nondeterministic** when it can allow several next steps for the same input. For a fixed sequence of inputs, a deterministic machine therefore exhibits a single behavior. A nondeterministic machine exhibits a set of behaviors, so its computation tree branches even when the inputs are fixed [25, slides 29 and 36].

The planning agent is deterministic in this sense. Given a configuration and an input, its guards select one transition. Its computation tree still branches, because the inputs can differ: the model can return a different reply, and a check can pass or fail. Every branch in the planning agent's tree is a point where an input can differ.

A deterministic machine is a different notion from the deterministic environment of Lecture 2. The environment is deterministic because the same tool call on unchanged files returns the same result. Determinism of the machine is a property of its transition rule. The model's replies are inputs to the machine, and neither notion makes them repeatable.

Definition 1.3 (Trace)**.** "A trace is the record of inputs, states, and outputs in a behavior." [26, slide 8]

The state and the trace hold different information. The state holds the run's current information only. The trace records the run's inputs, states and outputs over time.

The runtime's trace, which Lecture 2 introduced as the log of events the runtime writes during a run, is a trace in the sense of Definition 1.3. It is restricted to the inputs, state values and outputs that the runtime chooses to log. An **event** is one entry that the runtime appends to the trace. Every event has one of the following kinds:

- `model_call`**.** The event records one model call, the node that made it, and the tool calls that the reply requested.
- `routing`**.** The event records a routing decision, its destination, and the `by` field of Lecture 2, which names whether the model or the runtime supplied the value that the routing function tested.
- `tool_request`**.** The event records a tool call that the model requested.
- `tool_result`**.** The event records the outcome of a tool call.
- `state_change`**.** The event records a change that a node made to the state, such as the results of the checks, or, after an exception escapes the run, the name of that exception (Section 3).
- `terminal`**.** The event states how the run ended, and it is normally the trace's last event (Section 3).

The events written when the planning agent submits a patch show what the trace contains. In the recorded run of Section 1.8, the controller's last reply requested no tools, and the trace then holds the following events, in order:

1. A `model_call` event records the controller's call, with an empty list of tool calls.
2. A `routing` event records the decision to go to `submit`, with `by` set to `model`, because the reply's lack of tool calls selected it.
3. A `state_change` event records the results of the three checks in `submit`, all of which passed.
4. A `routing` event records the decision to go to `validate`, with `by` set to `runtime`, because the check results selected it.

These events contain a recorded input (the reply's empty list of tool calls), recorded state values (the check results) and recorded decisions.

The trace matters because the state does not outlive the program. The state lives in the memory of the running program, and when the program exits, any state that was not saved is lost. The trace and the files the agent changed can remain. After a run, they are what is left as evidence of what happened in it. Section 2 discusses what saving the state itself would require.

1.7 The planning agent as an extended state machine

Figure 1 draws the planning agent as an extended state machine, with each arrow labeled by its guard and set action. Output actions are omitted. In the labels, *calls* stands for the call count, the variable `model_calls`, and *budget* for the call budget of 40. The arrow from `START` carries only a set action, and the other arrows without a slash carry only a guard. The label "a check fails / append the check feedback" belongs to the dashed arrow from `submit` up to `controller`. We read the machine in parts: the forward path that an accepted run follows, the feedback edges that return to an earlier location, and the exits that end a run.

The forward path. An accepted run moves from `START` through `plan`, `controller`, `submit` and `validate` to `END`, with visits to `tools` along the way. The transitions on this path are the following:

1. **Start.** The arrow from `START` carries the initial set action: the status becomes `running` and the call count becomes zero.
2. **Plan.** If the call count is below the budget, the planner calls the model, stores the plan and counts the call.
3. **Controller.** If the call count is below the budget, the controller calls the model, appends the reply and counts the call. If the reply has tool calls, the run goes to `tools`. If it has none, the run goes to `submit`.
4. **Submit.** If all three checks pass, `submit` stores the check results and the run goes to `validate`.
5. **Validate.** If the call count is below the budget, the validator calls the model, stores its verdict in the status and counts the call. The run then ends.

The plan is stored in the state, and it reaches the controller as a message (Lecture 2, Section 4). The controller's model reads it. No routing function reads it. While the status is `running`, the routing after the controller depends only on whether the reply requests tools. The plan's subgoals are therefore carried out through the control locations already in the graph. No location corresponds to a subgoal, and no guard tests one.

The checks in `submit` are a gate in front of the validator. Only a patch that passes all three reaches `validate`, and only the validator can end a run with an accepted patch.

The feedback edges. A **feedback edge** sends execution back to a location that the run has visited earlier. Lecture 2 called such an edge a back edge. Figure 1 draws the planning agent's feedback edges dashed:

- **Tool loop.** The tool loop returns execution from `tools` to `controller` when there is no stall, or when the plan has already been revised. Its set action appends the tool results to the observations. This edge gives every observation back to the model, as in the reactive loop of Lecture 2. After a revision, a run that stalls again also takes this edge, and the controller's check then ends the run.
- **Revision.** A revision returns execution from `tools` to `plan` after the first stall. Its set action appends the observations and stores the reason for the stall, and the planner then writes a new plan.
- **Repair.** A repair returns execution from `submit` to `controller` when a check fails. Its set action adds the reasons for the failure to the messages, so the controller's model can write another patch.

Besides the tool loop, two feedback edges respond to failures: revision and repair. Section 3 gives each feedback edge its trigger, the information it passes back, and the bound on how often it can be taken.

The stall check that triggers a revision refines the stall check of Lecture 2. It reads the last three observations since the latest revision of the plan, or since the start of the run if there has been none. It reports a stall when all three are identical and none of them records a successful edit. A successful edit changes the files, so three observations that include one do not count as a stall.

Until the plan has been revised, the `tools` node runs this check after recording each new observation. On the first stall, it stores the reason in the variable `failed_step`, and the routing after `tools` sends the run to `plan`. After revising the plan, the planner stores the number of observations at that moment in `replanned_at`. Later stall checks start from that position, so the stall that caused the revision cannot count against the revised plan (Section 2). After the revision, the `tools` node no longer runs the check. The controller runs it before each of its model calls, so the controller detects a second stall.

The exits. A run reaches `END` through one of the following exits, and each sets the status on the way:

- **The budget.** Before each model call, `plan`, `controller` and `validate` compare the call count with the budget. If the count has reached 40, the node makes no call and the run ends with `budget_exhausted`. Lecture 2 described this check only for the controller. Every node that calls a model performs it.
- **A second stall.** If the run stalls again after a revision, the controller's check before its next call ends the run with `no_progress`.
- **The verdict.** The validator's verdict, `accepted` or `rejected`, becomes the status, and the run ends in both cases.

Each exit leads to a terminal configuration: the location `END`, with a status that records why the run ended. Section 3 examines what each ending establishes about the run.

1.8 A run as a trajectory

A trajectory is the path a run follows through the computation tree. Lecture 2 used the word for the record of the steps a run took. The computation tree makes the idea precise: a trajectory is one path from the root, and each tree node on it is the configuration on one visit to a control location. The behavior of a run is its sequence of steps, its trajectory is that sequence drawn as a path in the computation tree, and its trace is the record of the inputs, states and outputs along it.

We follow a recorded run of the planning agent on the worked alert of Lecture 2, a possible SQL injection in one test case of OWASP BenchmarkPython. The run took the forward path with four rounds through the tools. A round is one pass from the controller through `tools` and back, which is one turn of the loop in the terms of Lecture 2:

1. The planner wrote the plan.
2. The controller requested a tool four times, and `tools` ran it each time: `read_file`, `search`, `read_file` and `edit`.
3. On its fifth visit, the controller replied without tool calls, and the run went to `submit`.
4. All three checks passed, and the run went to `validate`.
5. The validator accepted the patch, and the run ended with the status `accepted`.

The run made seven model calls: one in `plan`, five in `controller` and one in `validate`.

Figure 2 places this run in the computation tree. Each tree node shows a control location and the number of model calls completed on reaching it. The blue path is the recorded trajectory, and its middle box stands for two more rounds through the controller and `tools`, drawn as one box. The tree branches at each visit to the controller (tool calls or none), at `submit` (all checks pass or a check fails) and at `validate` (accepted or rejected). A visit to `tools` can also branch, because a stall sends the run to `plan`. The figure does not draw that branch. The grey branch that leaves the controller's fifth visit shows a run that keeps requesting tools until the call count reaches the budget. Each ⋯ marks a part of the tree that the figure leaves out.

The machine of Figure 1 and the tree of Figure 2 answer different questions:

- **The machine.** It shows which transitions are permitted. It has one node per control location, so the controller appears once.
- **The tree.** It shows which sequences of transitions are possible from the initial configuration. It has one tree node per visit, so the controller appears five times on the recorded path, each time with a different configuration.

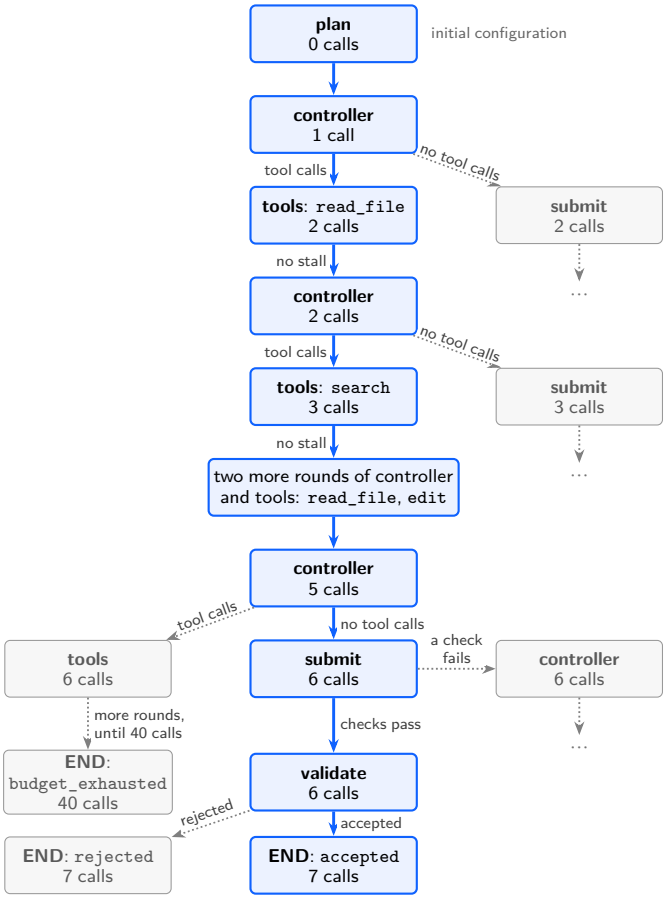


Figure 2: Part of the planning agent's computation tree. The blue path is the trajectory of the recorded run, and the grey nodes are configurations that other inputs would have reached. Each tree node shows its control location and the number of model calls completed on reaching it.

Another run on the same alert can follow a different path through the same tree. The graph and its guards are the same in every run. The model's replies are inputs, and a different reply selects a different child.

1.9 What the trace records and what we infer

The runtime's trace records selected inputs, outputs and state values, so some facts about a run are recorded and others must be inferred. The trace omits the following information:

- **Tool output.** A successful `tool_result` event records the length of the result in characters. It omits the text that the tool returned.
- **Some routing decisions.** The trace has no routing event for the moves from `tools` to `controller` and from `plan` to `controller`. We infer these moves from the controller's next `model_call` event and the transition rules.
- **The call count.** The trace does not record `model_calls`. We reconstruct it by starting at zero and adding one for each `model_call` event.

Every claim about a run must therefore say whether the trace records it directly or whether it is inferred from recorded events and the transition rules. Some facts are neither recorded nor inferable. In the recorded run of Section 1.8, the distinction applies as follows:

- **Recorded.** The results of the three checks are recorded in a `state_change` event, and the decision to go to `validate` in a `routing` event.
- **Inferred.** The call count of seven is inferred by counting `model_call` events. Each return from `tools` to `controller` is inferred from the controller's next call.
- **Absent.** The text that each `read_file` returned is not in the trace. A claim about what the model read cannot rest on the trace alone.

An inference depends on the transition rules it uses and on the events it starts from. If a rule is wrong or an event is missing, the inference can be wrong. Section 3 examines endings in which the event that would settle how a run ended is missing.

Key points

- A state machine is a simplified description of an agent that omits the prompts, the model and the tool code, and specifies states, inputs, outputs, an initial configuration, a transition rule and, for a workflow meant to end, terminal configurations.
- The state is the information retained between steps that can affect the next step, and model replies, tool results and check results are inputs that arrive during a step and select among the transitions the state permits.
- An extended state machine pairs finitely many control locations with variables, and a configuration is a location together with the value of every variable, so an unbounded message list can give one location infinitely many configurations.
- A transition $(q,v) \xrightarrow{e;g/out;set} (q',v')$ is allowed when its guard g holds for the variables and the input e, and it then emits out, applies set and moves to q'.
- The planning agent is deterministic because its guards select one transition for each configuration and input, so its computation tree branches only where an input can differ, and a run's trajectory is one path through that tree.
- In the planning agent, the plan reaches the controller only as a message that no guard reads, the feedback edges are the tool loop, revision and repair, and a run ends when the call count reaches the budget of 40, at a second stall, or with the validator's verdict.
- Unsaved state is lost when the program exits, and the trace that remains records only selected inputs, outputs and state values, so every claim about a run must say whether it is recorded or inferred from recorded events and the transition rules.

2 State, records and reducers

Section 1 described the planning agent as an extended state machine, in which a configuration pairs a control location with the current value of every variable. This section shows how the runtime stores those values, how one step changes them, and how long each stored value stays valid. It also separates the stored values from what a model call receives and from what survives the process, proves that the call budget holds at every step, and shows how the graph is built. The routing and stopping rules of Section 3 read these stored values, so they rest on the mechanisms described here.

2.1 State records and schemas

A **state record** stores the information that an agent retains between steps as named fields with their current values. A **field** is one named entry of the record. Section 1 called the retained information the state and called its parts variables, so the record is the stored form of the state and each field holds the value of one variable. For example, the field `model_calls` with the value 2 records that two model calls have completed.

The runtime keeps the control location and the variable values of a configuration in different places. The runtime itself tracks the control location, which identifies the computation to run next. The state record holds the values of the variables, such as the messages exchanged with the model and `model_calls`. The graph of the agent describes control flow, that is, which computation may run next. The record supplies the values that those computations read.

Each run has its own record, which holds the current values for

that run. Every node of the run reads the same record, so the record is the state that the nodes share.

The fields of the record are declared in a state schema, which, as in Lecture 2, lists the fields the state contains and their types. Our agents are built with LangGraph, a library for writing programs as graphs of nodes and edges. LangGraph documents a `TypedDict` class as the main way to write a schema [9]. A `TypedDict` is a Python dictionary type with a fixed set of keys and a declared type for the value of each key [12]. Each key of our schema names one field.

Our schema is the class `AgentState`. LangGraph builds a graph with its class `StateGraph`, and the call `StateGraph(AgentState)` creates a graph whose shared state has the fields that `AgentState` declares.

The schema and the record play different roles. The schema declares the names and types of the fields, and it is the same for every run. The record holds values, and they change from step to step. A node never returns a whole record. It returns only the fields that it changes (Section 2.2). Once those fields are merged into the record, routing functions read the updated record to select the next node (Section 3).

2.2 Partial updates

A node changes the record by returning a partial update, which the runtime merges into the record one field at a time. A **partial update** is a mapping from the fields that a node changes to the values it returns for them.

A **reducer** is the rule that merges the value stored in a field with the value that a node returns for that field. Every field has a reducer. A field whose declaration names none takes the default reducer, which replaces the stored value with the returned one [9]. Section 2.5 describes the kinds of reducer our agents use.

One step proceeds as follows:

1. The runtime passes the current record, written s_t, to the node.
2. While it runs, the node receives an input, written e_t: a model reply, a tool result or a check result.
3. The node returns a partial update, written u_t.
4. For each field in u_t, the runtime calls that field's reducer. The stored value is the reducer's left argument, and the returned value is its right argument.
5. The merged record is the next state, s_{t+1}.

Figure 3 draws these steps. The dashed arc carries the stored value from s_t straight to the reducer, so the left argument never passes through the node.

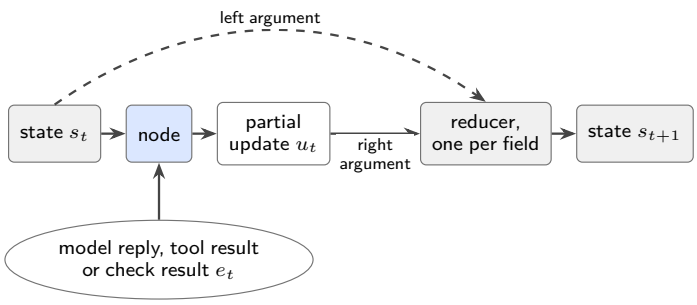


Figure 3: One step of a run. The node reads the stored state and an input, and returns a partial update. For each field in the update, the field's reducer merges the stored value (left argument) with the returned value (right argument), and the result is the next state.

For a field f with reducer r_f, where $s_t[f]$ is the value of f in the record s_t, the merge follows one rule:

$$s_{t+1}[f] = \begin{cases} r_f\big(s_t[f], u_t[f]\big) & \text{if } u_t \text{ contains } f, \\ s_t[f] & \text{otherwise.} \end{cases}$$

A field omitted from the partial update keeps its stored value. A node therefore never has to copy the fields it leaves unchanged. It also cannot erase a field by leaving it out.

A node controls only the value it returns. The field's reducer, which the schema declares, fixes how that value combines with the stored one. The one exception is an explicit overwrite, which bypasses the reducer (Section 2.5).

In the terms of Section 1, the partial update and the reducers together carry out the set action of the transition, the update of the variables. The run then moves to its next control location, which a routing function selects by reading the merged record (Section 3).

2.3 The controller's return sites

The controller ends every visit with a partial update that either stops the run with a status or records one model call: the reply, the completed call and the reply's token usage. We call each `return` statement of a node a **return site**, and each return site gives the runtime one partial update. The controller's return sites, in the order its code reaches them, are the following:

- **Budget.** If the call budget is spent, the controller returns only the field `status`, set to `budget_exhausted`.
- **Stall.** If the stall check finds that the run has stalled, the controller returns only `status`, set to `no_progress`.
- **Model call.** Otherwise, the controller calls the model once. It returns the reply in `messages`, one completed call in `model_calls`, and the reply's token counts in `usage`.

The first test is the **admission check**, the comparison of `model_calls` with the budget before each model call (Section 2.4). Both stopping tests run before the model call, so a run that must stop makes no further call. A routing function then reads the status and ends the run (Section 3).

Figure 4 applies these return sites to the reactive loop of Lecture 2, Section 3. Its record is smaller, with ten fields, and its controller has the same return sites in the same order as the planning agent's. After a model call, the update changes `messages`, `model_calls` and `usage`. The reducer of `messages` appends the reply, the reducer of `model_calls` adds one, and the reducer of `usage` adds each token count to its running total. A stopping return site changes only `status`. That field declares no reducer, so it takes the default, and the returned string replaces the stored one.

The remaining fields, `alert`, `repo`, `commit`, `workspace`, `observations` and `candidate_patch`, are absent from every update the controller returns. They keep their stored values after every visit to the controller.

Listing 2 shows the planning agent's controller. The listing is illustrative and leaves out the event that the controller writes to the trace.

This controller differs from the reactive loop's in its stall check, which reads only the observations recorded since the latest revision of the plan, or since the start of the run if the plan has not been revised. The function `planning_window` selects them (Section 2.6). The tools node detects the first stall and sends the run to the plan node. In the planning agent, this return site therefore ends the run only when the run stalls again after a revision.

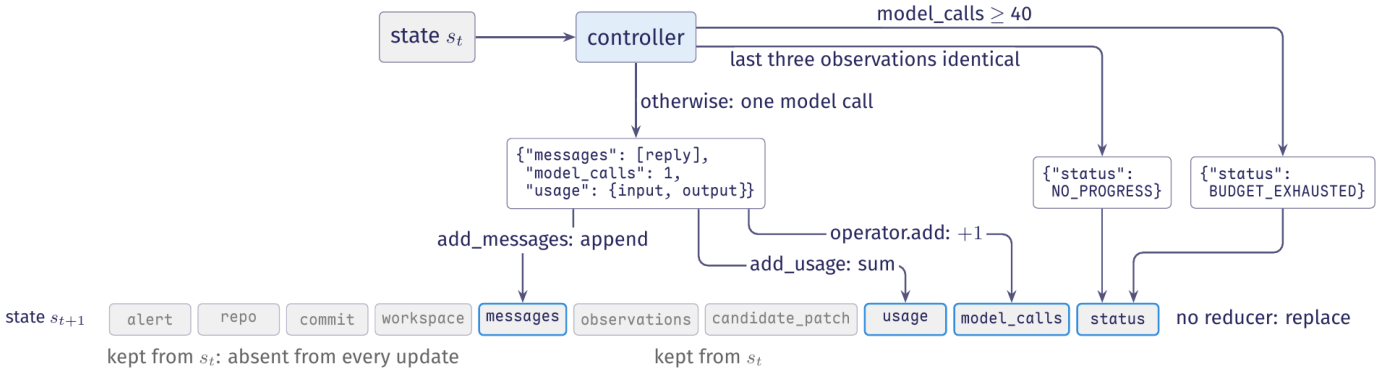


Figure 4: The return sites of the reactive loop's controller, applied to its ten-field record. Each of the two stopping paths returns only a status. The path through a model call returns the reply, one completed call and the reply's token usage (shown with its input and output counts), and each of those fields is merged by its reducer. Fields in grey appear in no update and keep their stored values.

Listing 2: Illustrative code for the planning agent's controller node. Each `return` is one return site.

```
def controller(state):
    stop = admission_check(state)       # budget
    if stop:
        return stop
    if stalled(planning_window(state), limit=3):
        return {"status": NO_PROGRESS}  # stall
    reply = call_model(state["messages"])
    return {"messages": [reply],        # model call
            "model_calls": 1,
            "usage": usage_of(reply)}
```

The first return site passes on whatever the admission check returns (Listing 3). When the budget is spent, the check returns a status update that ends the run. Otherwise it returns `None`, and the controller goes on to the stall check.

The second return site applies `stalled` to the observations that `planning_window` selects. With `limit=3`, it reports a stall when the last three of them are identical and none records a successful edit. The last return site reports one completed call as `"model_calls": 1`. The reducer adds this one to the stored count, so a stored count of 2 becomes 3.

2.4 Model context, runtime state and durable state

Information about a run exists as model context, runtime state and durable state, which differ in what they hold, who reads them and how long they last. The **model context** is what one model call receives: the instructions, a selection of the history, and observations. Lecture 2 called it the model's context.

The **runtime state** is the set of fields that the running process retains between steps, including counters that the model never sees. It is the state record of Section 2.1, held in the memory of the running process. We say runtime state when we contrast the record with the model context and the durable state.

The **durable state** is state that outlives the process, in a form that another process can load to resume the run. Table 2 compares the three. Its last column shows the difference that matters for resuming: only the durable state can be loaded by a new runtime process.

For each model call, the runtime builds the model context from the runtime state. The controller passes the conversation stored in `messages` to the model (Listing 2). The validator's context is built from other fields: the alert, the patch, the check results and the observations (Lecture 2, Section 4). It never includes the conversation. A model context is therefore a selection from the runtime state, made separately for each call.

To resume a run after its process exits, the runtime must have saved enough state to reconstruct the execution. A **checkpointer** is the component that saves the runtime state as checkpoints [10]. Lecture 2 called such saved progress a checkpoint. A checkpointer that writes its checkpoints to a file or a database produces durable state, which a new process can load to continue the run. One that keeps them in memory loses them when the process exits. Our graphs are compiled without a checkpointer and keep their state in memory. When the process exits, the record is lost (Section 1).

Table 2: Model context, runtime state and durable state: what each holds, and who reads it.

State or context	What it holds	Who reads it
Model context	The instructions, selected history and observations supplied to one model call	The model
Runtime state	The fields retained between steps, including counters that the model never sees	Nodes, routing functions and limit checks
Durable state	State that survives the process, in a form another process can load to resume the run	A new runtime process

The trace survives the process, but a new process cannot rebuild the record from it. The trace records selected events and omits values that resumption needs, such as the text that each tool returned, which no event records (Section 1).

A saved record alone is also not enough to resume. The workspace files lie outside the state, so a resumed run needs files that match the saved record. For example, if the record holds a patch that was applied before the exit, the files must already contain that patch. A resumed run also needs the pending work, that is, the steps that were scheduled but had not finished when the process exited. A later unit of the course covers saved state, pending work and the workspace.

The call budget shows why the distinction between runtime state and model context matters. None of the planning agent's prompts mentions the budget, so the budget is in no model context. The admission check enforces it from the runtime state instead (Listing 3). The planning agent runs this check before every model call, in each node that calls the model: `plan`, `controller` and `validate`.

Listing 3: Illustrative code for the admission check, which runs before every model call.

```
def admission_check(state) -> dict | None:
    if state["model_calls"] >= budget:   # budget = 40
        return {"status": BUDGET_EXHAUSTED}
    return None                          # call allowed
```

The check works even though the model never receives the budget. It is code in the runtime, and it compares a counter that the model cannot write with a limit that we fix before the run starts. Whatever the model writes, a call is issued only while `model_calls` is below the budget.

2.5 Reducers

A field's reducer decides how a returned value combines with the stored one, and our agents use reducers that append, replace or accumulate. The kinds of reducer work as follows:

- **Append.** The reducer adds the returned items to the end of the stored list, so later nodes can read the full history. The fields `messages` and `observations` use append reducers.
- **Replace.** The reducer stores the returned value as the field's new value, as when the plan node writes a revised plan. A field declared without a reducer takes replacement as its default [9].
- **Accumulate.** The reducer combines the returned contribution with the stored total, for example by adding 1 to `model_calls`. The field `usage` accumulates token counts in the same way.

The schema attaches a reducer to a field with the form `Annotated[T, fn]`. `Annotated` is the standard Python way to attach extra information to a type. Here `T` names the field's type, and LangGraph reads `fn` as the field's reducer (Listing 4). A field written with its type alone, such as `status: str`, declares no reducer and takes the default, which replaces the stored value.

Listing 4: Illustrative code for part of the state schema, with the reducer of each field.

```
class AgentState(TypedDict, total=False):
    # other fields omitted
    messages: Annotated[list, add_messages]
    observations: Annotated[list, operator.add]
    usage: Annotated[Usage, add_usage]
    model_calls: Annotated[int, operator.add]
    status: str
```

Each field in the listing has the following reducer:

- **`messages`.** The reducer `add_messages` appends the returned messages to the conversation. A returned message whose ID matches a stored message replaces that message instead.
- **`observations`.** The reducer `operator.add` joins two lists, so it appends the returned observations.
- **`usage`.** The reducer `add_usage` accumulates. It keeps four separate token counters and adds each returned count to the counter of the same kind. One counter holds cache reads, the input tokens that the provider reuses from an earlier call, so they stay distinct from fresh input tokens.
- **`model_calls`.** The same `operator.add` adds two integers here, so it accumulates the count of completed calls.
- **`status`.** The field declares no reducer, so it takes the default, and an update replaces the stored string. If a node returns no status, the runtime keeps the previous one.

The words `total=False` in the class header declare that a dictionary of this type may leave keys out [12]. A partial update, which holds only some of the keys, therefore still has the declared type.

Returning `[]` to an append reducer adds zero items, so the stored list keeps its contents. Clearing a list such as `observations` therefore requires an explicit overwrite, which bypasses the reducer and replaces the stored value. LangGraph provides the wrapper `Overwrite` for this. A node that returns a value wrapped in `Overwrite` replaces the stored list with that value. A reset of this kind should be recorded as its own trace event. A reader of the trace otherwise assumes that the list only grows, and cannot tell that earlier entries were discarded.

2.6 Field lifetimes

A **field lifetime** is the interval during which later nodes may rely on a field's stored value. The lifetime ends when a later step makes that value obsolete, for example when a revision of the plan makes the old plan obsolete. A reducer decides only how a returned value is merged. It does not decide how long later nodes may rely on that value.

Table 3 defines the lifetimes. In our two agents, the identity fields are `alert`, `repo`, `commit` and `workspace`. The run fields are `messages`, `observations`, `usage` and `model_calls`. The current fields are `candidate_patch`, `status`, `plan`, `checks` and `validation`.

A **handoff field** holds a reason that one node leaves for another and is cleared after use, here `failed_step`. A **window field** holds an index where a window of a history starts, here `replanned_at`. An attempt is one try at producing an accepted patch. No field of either agent has the attempt lifetime.

Figure 5 sets the two schemas side by side. The shading of each field gives its lifetime from Table 3, and a plus sign marks the fields that the planning agent adds: `plan`, `failed_step`, `replanned_at`, `checks` and `validation`.

The plan node writes `plan`, the submit node writes `checks`, and the validate node writes `validation`. Each of these fields holds the latest value until its writer replaces it. The tools node writes `failed_step` for the plan node to read and clear. The index `replanned_at` marks where the observations made under the current plan begin.

Table 3: The field lifetimes, with what each one means for the nodes that read and write the field.

Lifetime	Definition and use
Identity	Identity fields identify the task, the target and the workspace, and they remain unchanged.
Run	Run fields retain histories and counters that grow throughout the run.
Current	Current fields hold the latest plan, candidate, check result or status until the writer replaces them.
Handoff	One node writes a reason for invoking another node. The receiving node reads that handoff field and clears it after acting.
Window	An index field marks where a selected window of history begins. Checks read from that index onward while the complete history remains stored.
Attempt	An attempt field holds the results of one attempt. Nodes clear it before a new attempt, so later steps cannot treat old results as current.

The attempt lifetime is absent because neither agent starts a new attempt. In the planning agent, a rejection by the validator ends the run, and the reactive loop has no validator. In both agents, a patch that fails the checks returns to the controller for repair, but no field is cleared, and the run continues as a single attempt. Neither schema therefore needs fields that must be cleared before a new attempt.

Section 3 discusses a proposed new attempt after a rejection. If it were added, fields that hold one attempt's results, such as the plan and the candidate patch, would become attempt fields.

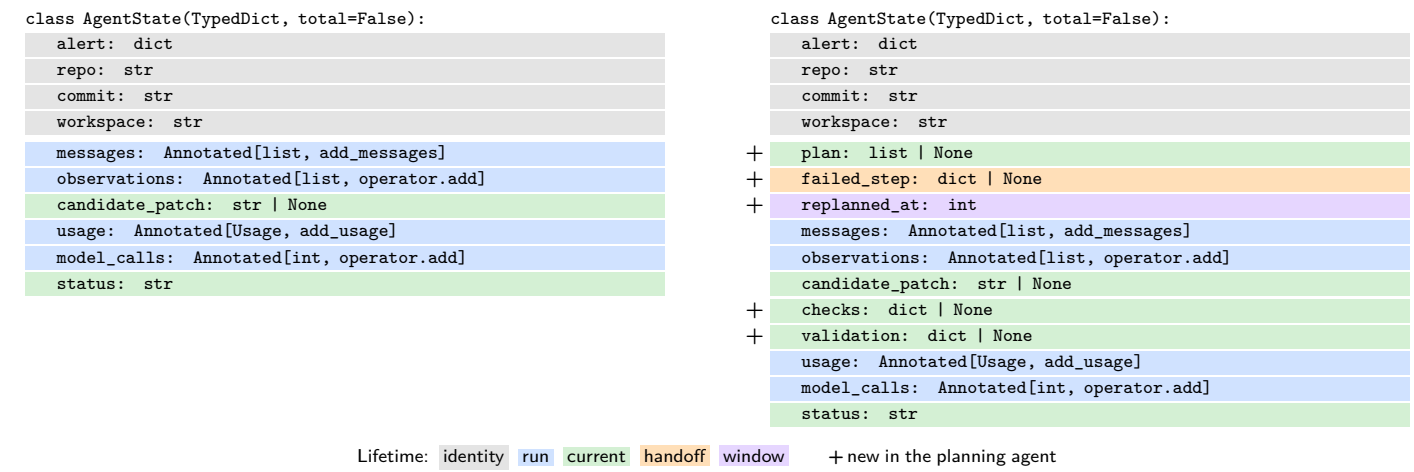


Figure 5: The state schemas of the reactive loop and the planning agent. The shading of each field gives its lifetime, and a plus sign marks the fields that the planning agent adds.

Listing 5: Illustrative excerpts from the tools node and the plan node: the handoff in `failed_step` and the window in `replanned_at`.

```
# tools node: hand off to plan on the first stall
update = {"observations": new}  # tool messages omitted
window = state["observations"] + new  # not merged yet
if (not state.get("replanned_at")
      and stalled(window, limit=3)):
   update["failed_step"] = {...}   # the reason
# plan node: revise, clear the handoff, open a window
# (the admission check that runs first is omitted)
failed = state.get("failed_step")
steps = call_planner(state, failed)   # one model call
update = {"plan": steps, "failed_step": None,
          "model_calls": 1}   # messages, usage omitted
if failed:
   update["replanned_at"] = len(state["observations"])
```

The runtime merges updates according to reducers. It does not enforce lifetimes, so node code must enforce them, by retaining, replacing or clearing each field when required. A reducer sees only the stored value and the returned value. It cannot know that a revision has happened or that a reason has been acted on.

Run fields must keep their values across attempts, which matters once a new attempt is added (Section 3). The call count shows why. A counter that limits the whole run must keep its total. If `model_calls` were reset to zero at a new attempt, later calls would ignore the calls already spent, and the budget would no longer bound the run. Attempt fields are the opposite case. They would be cleared before a new attempt because their old results no longer apply.

The handoff field and the window field of the planning agent work together when the run stalls (Listing 5). When the tools node detects the first stall, it stores the reason in `failed_step`. The plan node uses that reason to revise the plan, then clears `failed_step` and sets `replanned_at`.

The tools node tests the stored observations together with its new ones, because the runtime merges its update only after the node returns (Section 2.2). Its update always holds the new observations, and it gains `failed_step` only on the first stall.

Clearing `failed_step` removes the request for the revision just completed. If the field remained set, the routing function after the tools node, `after_tools`, would select the plan node again on the next tools step. The test `not state.get("replanned_at")` also limits the handoff to one revision, because the index is set once the plan has been revised.

The plan node sets `replanned_at` to the number of observations stored when the revision completes. The slice `observations[replanned_at:]` therefore starts with the observations made under the revised plan. The stall check reads only this window, while the earlier observations remain in the full history. Lecture 2 called `replanned_at` a high-water mark.

A window field lets a check ignore old entries of a run field without clearing that field. The list `observations` has an append reducer, so clearing it would need an explicit overwrite (Section 2.5). It would also remove evidence that later nodes read, such as the validator, which judges the patch from the observations.

2.7 The call budget as an invariant

The call budget bounds the number of completed model calls because a property of the record holds at every step. An **invariant** is a property that holds in every reachable configuration of a machine. The standard proof first shows that the property holds in the initial configuration. It then shows that every transition from a configuration where the property holds leads to a configuration where it still holds. The property then holds in every configuration that a run can reach [16, Theorem 2, p. 5].

The invariant of the call budget is that `model_calls` counts the completed model calls exactly and never exceeds the budget B, which is 40 in our agent:

$$\texttt{model_calls} = \text{number of completed model calls} \leq B.$$

In the initial configuration, `model_calls` is 0 and no call has completed. The argument that every transition keeps the property true covers steps that call the model (admission and counting) and steps that do not (preservation):

- **Admission.** Each node that calls the model checks that the stored call count is below the budget before it issues the call. A call therefore starts only when the count is at most $B-1$.
- **Counting.** After a call completes, the node returns `"model_calls": 1`, and the reducer adds one to the stored count. The count rises by one for each completed call, and because the call started at $B-1$ or less, the count reaches at most B.
- **Preservation.** A step without a model call leaves the count unchanged, because the field is absent from its partial update. No transition resets or decreases the count.

The reducer only accumulates the calls that nodes report. The property therefore depends on every caller. Each node must check the budget before calling and must report each completed call exactly once. A node that called the model without the admission

check, or reported a call twice, would break the invariant, and the reducer could not detect it.

The argument covers completed calls only. A call that is interrupted, for example because the process is stopped while the call is in progress, may consume tokens without being counted. Its node never returns an update, so `model_calls` never includes it. The invariant bounds the completed calls of a run. It does not bound everything the run spent. Section 3 uses the budget to argue that a run ends.

2.8 Building and compiling the graph

The graph is built by declaring its schema, its nodes and its edges, and then compiling the declarations into a graph that the runtime can execute (Listing 6). The call `StateGraph(AgentState)` creates one **channel** per field of the schema. A channel is the runtime's store for one field. It holds the field's value and applies each update with the field's reducer. For a field that declares no reducer, the channel replaces the stored value, as the default reducer does [11].

Listing 6: Illustrative code that builds and compiles the planning agent's graph, with most declarations omitted.

```
graph = StateGraph(AgentState)   # one channel per field
graph.add_node("plan", plan)
graph.add_node("tools", tools)
# other nodes and edges omitted
graph.add_edge(START, "plan")
graph.add_conditional_edges("tools", after_tools,
    {"controller": "controller", "plan": "plan"})
graph.add_edge("validate", END)
app = graph.compile()
```

The declarations do the following:

- **`add_node`.** It associates a node name with the function that the runtime will call for that node. The function receives the state and returns a partial update.
- **`add_edge`.** It names the node that always runs after a source node finishes. The edge from `START` begins every run at the plan node, and the edge to `END` ends the run after the validator.
- **`add_conditional_edges`.** It attaches a routing function to a source node, together with a mapping from each value the function can return to the node that runs next. After the source node's update is merged, the runtime calls the routing function, which reads the updated record and selects one of these successors. After the tools node, the destination is the plan node after the first stall, and the controller otherwise.
- **`compile`.** It checks the declared connections of the graph and returns a graph that the runtime can execute.

The runtime fixes the order of operations in a step. The node runs and returns its update, the channels merge it, and only then does a routing function read the record. A routing decision therefore always sees the values that the node just returned.

2.9 Designing a state graph

A **state graph** is a graph whose nodes read and update one shared state record, like the one built in Section 2.8. We design one by declaring each field at the same time as the node that writes it, in the following steps:

1. **Specify the nodes** and the rules that select which node runs next.
2. **Declare fields** for the values that nodes and routing functions read.
3. **Name the node** that writes each field, and say when later nodes must stop using its old value.
4. **Choose each field's lifetime,** and then choose whether its updates append, replace or accumulate values.
5. **Write the initial record** with a value for every field.
6. **Make each node** return a partial update that contains only the fields it proposes to update.

The field `failed_step` of the planning agent shows the steps applied to one field. The plan node and the routing function after tools read it, so it is declared. The tools node writes it, and its value is obsolete once the plan node has revised the plan. Its lifetime is therefore a handoff, and its update replaces the stored value.

The initial value of `failed_step` is `None`. When the tools node detects the first stall, it adds the field to the partial update that already holds its new observations and messages, and it leaves every other field out of that update. The plan node returns `None` for the field once the revision is done.

Each field retains a value that a later computation needs to read. A field that no computation reads should be left out of the schema.

Key points

- A state record stores the information an agent retains between steps as named fields, the runtime tracks the control location separately, and the schema declares the fields' names, types and reducers.
- A node returns a partial update, each returned field's reducer merges the stored value (left argument) with the returned value (right argument), an omitted field keeps its value, and routing then reads the merged record.
- In order, the controller's return sites give the status `budget_exhausted` when the budget is spent, the status `no_progress` when the run has stalled, and otherwise the reply, one completed call and its token usage.
- The model context is the part of the runtime state selected for one model call, and the durable state is what a checkpointer saves to a file or a database so that another process can resume the run.
- A reducer appends, replaces or accumulates, and since returning an empty list to an append reducer adds nothing, clearing a list such as `observations` needs an explicit overwrite, recorded as its own trace event.
- A field's lifetime is the interval in which later nodes may rely on its value, and node code must enforce it, as the plan node does when it clears the handoff field `failed_step` and sets the window field `replanned_at`.
- Because each node that calls the model runs the admission check before the call and reports each completed call once, `model_calls` never exceeds the budget, which no model context contains, but the bound covers completed calls only, and an interrupted call can consume tokens without being counted.

3 Routing, feedback and stopping

The state record of Section 2 holds every value that the planning agent keeps between steps. This section shows how the merged record selects the next step, sends execution back to an earlier location when work has to be redone, and ends the run. Every return to an earlier location adds work to the run, and every ending is later read from the trace, so we need to know what these mechanisms guarantee. We ask when a run is certain to stop, and what its trace can establish about how it stopped.

3.1 Routing functions and conditional edges

After a node runs, the routing function attached to it reads the merged record and selects the next control location. As in Lecture 2 and Section 1, a routing function is code that we write: it reads the state and returns the name of the node that runs next. In the planning agent, every node except `validate` has a routing function. The edge from `validate` to `END` is unconditional, so the run always ends after the `validate` node. Its status is the verdict, or `budget_exhausted` when the admission check stopped the validator's call.

A step in which the model is called passes through a cycle of proposal, record and decision:

- **Proposal.** The model emits a tool call, a plan, a candidate patch or a verdict. We call this output a proposal because it decides nothing by itself.
- **Record.** The runtime merges the node's partial update into the state record, using each field's reducer (Section 2). It also appends events to the trace that describe what happened.
- **Decision.** The routing function tests fields of the merged record. The guards and limits that the programmer wrote determine which destination it returns. After the `validate` node, the destination is always `END`. The status records the verdict, or `budget_exhausted` when the admission check stopped the call before a verdict.

Lecture 2 described a conditional edge as an edge that is taken only when a predicate over the state selects it. In the vocabulary of Section 1, each conditional edge is one transition, and its predicate is that transition's guard, a condition on the merged record. All the conditional edges that leave one control location share one routing function. The function tests their guards and returns one destination.

A guard can test any field of the merged record. This includes the value the latest step wrote, such as the model's reply. It also includes values retained from earlier steps, such as the number of model calls made so far. Another example is the handoff field `failed_step`, which the tools node sets on the first stall and the routing function after `tools` tests (Section 2). A routing decision therefore depends on the history of the run, as far as the record retains it.

A routing function stops the run by returning `END`. The returned name says only that the run is over. The `status` field of the record says why it stopped.

The trace records routing decisions as routing events (Lecture 2). It has no routing event for the moves from `tools` and from `plan` to the controller, which are inferred (Section 1). A routing event's `by` field names the source of the value that the guard tested:

- **By the model.** The field holds `model` when the tested value came from the model's output, for example the presence of a tool call in its reply.
- **By the runtime.** The field holds `runtime` when the tested value came from a check, such as a failed submit check, or from a limit, such as an exhausted budget. It also holds `runtime` for a default, a value that the runtime supplies in place of a model output.

In both cases the programmer defined the edge and its guard. The `by` field records who supplied the tested value. It does not record who wrote the rule that tested it.

The routing function after the controller, `after_controller`, shows how the model's reply affects routing at that location. Section 1 gives its code in Listing 1, and it is the same function as in the reactive loop of Lecture 2, Section 3. It makes two tests, in this order:

- **The status test.** If the status is no longer `running`, a limit or a check has stopped the run, and the function returns `END`. The runtime supplied the tested value.
- **The tool-call test.** For an active run, the function returns `tools` if the last reply contains tool calls, and `submit` otherwise. The model supplied the tested value.

The function reads the `tool_calls` field of the reply. It never reads the reply's text. Suppose that an active run receives a reply that claims the patch is correct and also requests a tool. The run goes to `tools`, because the reply contains a tool call, and the claim in the text has no effect. At the controller, the model can change the path of a run only by including tool calls in its reply or leaving them out. The guards we wrote decide where each choice leads.

3.2 The same graph, different paths

The compiled graph fixes which transitions are possible, and the record decides which of them a run takes. As in Lecture 2, the graph is compiled once, before any alert is read, and every run executes the same compiled graph. During a run, each routing function uses the current record to select a destination.

Every visit to a control location that has a routing function leads to a new routing decision. A run can visit the controller many times. At each visit, the routing function after the controller runs again on the record as it stands after that visit. The same location can therefore lead to different destinations within one run.

The computation tree of Section 1 shows this. The tree has a separate node for each visit to a control location. Each tree node represents the configuration of that visit: the location together with the values in the record. In the recorded run of Section 1, the first four visits to the controller produced replies with tool calls and routed to `tools`. The fifth produced a reply without one and routed to `submit`. The location, the graph and the routing function were the same at every visit. Only the record differed.

The same tree also contains the branches that this run did not take. At `submit`, a failed check would have sent the run back to the controller. At `validate`, the validator could have rejected the patch. At a later visit to the controller, the budget could have run out. The compiled graph permits all of these paths, and the values in the record select one of them.

3.3 Feedback edges and their bounds

Every feedback edge must specify its trigger, the information it passes back and its bound. A feedback edge, as Section 1 defines it, sends execution back to a location visited earlier in the run. Every return of this kind adds work to the run, so each part has to be stated before the edge is added to a graph:

- **Trigger.** The trigger is the condition on the record under which the edge is taken.
- **Information passed back.** This is what the earlier location receives on its return, together with the fields that are reset. It is what allows that location to act differently the second time.
- **Bound.** The bound limits how often the edge can be taken. It is either a limit of the edge's own or a limit on the whole run.

An edge without a bound could return execution to the same location indefinitely, and the run would never end.

The planning agent's tool loop, from `tools` back to the controller, returns every observation to the model. Besides the tool loop, two feedback edges respond to failures: revision and repair. A third, the new attempt, is proposed.

A **new attempt** is a restart of the work on the same alert after the validator rejects a patch, beginning again at `plan`. Before it starts, the run writes a **reflection**, a description of why the rejected attempt failed, and passes it to the new attempt. An **attempt counter** records how many attempts the run has made, and an attempt limit caps that number.

Table 4 sets the edges that respond to failure side by side. Its last column shows that each of them has a bound, either its own or a limit on the whole run.

Revision returns execution to `plan` after the first stall, and it replaces the stalled plan. A stall, as Section 1 defines it, occurs when the last three observations since the latest revision, or since the start of the run, are identical and none of them records a successful edit. The edge works as follows:

Table 4: The feedback edges that respond to failure: revision and repair in the planning agent, and the proposed new attempt. Each row gives the edge's trigger, the information it passes back with the fields it resets, and the limit on how often it can be taken.

Feedback edge	Trigger	Information and reset	Bound
Revision: tools to plan	The last three observations under the current plan are identical and include no successful edit.	The plan node uses the stall reason to revise the old plan. It clears the handoff field and limits later stall checks to the new plan's observations.	The runtime permits one revision per run. A second stall ends the run.
Repair: submit to controller	At least one of the three submit checks fails.	The runtime adds the reasons to the conversation. No fields are reset. Later submissions replace the patch and the check results.	The call budget limits the number of repairs.
New attempt (proposed): validate to plan	The validator rejects the patch, and the attempt counter is below the attempt limit.	The runtime keeps the reflection and clears the attempt fields: the plan, the patch and the check results.	The attempt limit caps the attempts, and the attempt counter survives resets. The call budget also applies.

- **Trigger.** The `tools` node detects the stall and stores the reason in the handoff field `failed_step` (Section 2). The routing function after `tools` sees the field set and returns `plan`.
- **Information and reset.** The plan node uses the stall reason to revise the old plan. It then clears `failed_step` and sets the window field `replanned_at`, so that later stall checks read only the observations made under the new plan.
- **Bound.** The runtime permits one revision per run. After a revision, the `tools` node no longer sets `failed_step`, so the routing after `tools` returns to the controller even when the run stalls again. The controller's stall check before its next call then ends the run with the status `no_progress`.

Repair returns execution to the controller with the reasons why a patch failed its checks, and it asks the controller for another patch. The `submit` node runs three checks: the patch applies, it touches the flagged line, and the rescan is clean.

- **Trigger.** At least one of the three checks fails.
- **Information and reset.** The runtime adds the reasons for the failure to the conversation, so the controller's next model call receives them. No fields are reset. A later submission replaces the patch and the check results.
- **Bound.** Repair has no limit of its own. The call budget limits the number of repairs, because every repair returns to the controller, and each later visit to the controller either ends the run or makes a model call.

In the planning agent, a rejection by the validator ends the run with the status `rejected`. The proposed new attempt would turn a rejection into a fresh start, and Figure 6 draws the proposed edge. In the figure, the oval labeled Alert is the start of the run, and one box stands for the controller, the tools and the submit node. Dotted elements are proposed, and the dashed loops are the existing revision and repair edges.

After a rejection, the run either writes a reflection, resets the attempt and returns to `plan`, or ends, depending on how many attempts it has made. We write a for the value of the attempt counter and A for the attempt limit. When the validator rejects a patch and $a < A$, the run writes a reflection and starts a new attempt. When $a \geq A$, the run ends with the status `rejected`.

With the new attempt, the plan, the candidate patch and the check results would become attempt fields, which hold the results of one attempt (Section 2). The reset before a new attempt clears them, so that no later step treats the rejected attempt's results as current. The reflection survives the reset, and so do the run's counters.

The attempt counter must count attempts across the whole run. If each reset erased it, the counter would read zero at the start of every attempt, and the runtime could not enforce a limit on the total number of attempts. The counter therefore has run lifetime, like `model_calls`. The call budget still applies across all attempts.

Every feedback edge must have its own limit or rely on a limit for the whole run. Revision has its own limit of one per run. Repair and the tool loop rely on the call budget, because both return to the controller, which either ends the run or makes a model call. The proposed new attempt has both, the attempt limit and the call budget.

3.4 Loop bounds and termination

A condition for success does not guarantee that a run stops, so termination needs an argument of its own. The argument answers the question Lecture 2 left open: when do the stopping rules guarantee that a run ends?

An **acceptance predicate** is the condition under which a run is declared successful. In the planning agent, a patch satisfies it when it passes the three submit checks and the validator accepts it. The **acceptance procedure** is the code that evaluates the predicate: the submit checks, followed by the validator's verdict.

A run that never produces an acceptable patch can keep cycling through its feedback edges unless something else stops it. The acceptance predicate therefore says nothing about whether a run ends.

The usual argument uses a **loop variant**, a nonnegative integer that decreases on every continuing cycle [1]. A **continuing cycle** is one pass around a loop of the graph, back to the location where it began, after which the run does not end. Because the variant cannot fall below zero, it bounds the number of cycles. A variant that starts at n allows at most n continuing cycles.

In the planning agent, the variant is $B - c$: the call budget B minus the number c of calls already made. With our budget of 40, it starts at 40. The argument rests on three conditions:

- **Nonnegative.** The admission check (Section 2) keeps $B - c$ nonnegative, because a node makes a model call only while $c < B$.
- **Decreasing.** Every continuing cycle makes at least one model call, so $B - c$ drops by at least one per cycle.
- **Each step returns.** Every step between two model calls returns or times out, so the run reaches its next model call or its end.

Under these conditions, a run completes at most B continuing cycles. The conditions bound the number of cycles. They do not by themselves bound how long a single step takes.

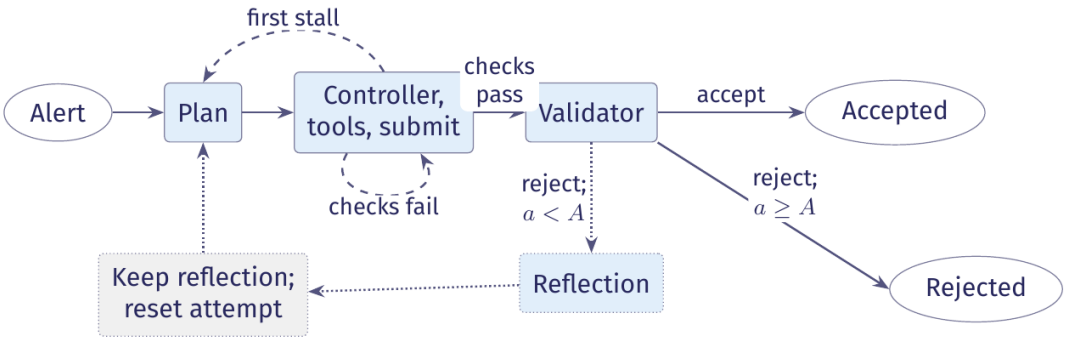


Figure 6: The proposed feedback edge for new attempts, from the validator back to the plan node. A rejection leads to a new attempt while a, the number of attempts so far, is below the attempt limit A, and ends the run otherwise.

The planning agent meets the decreasing condition. Every loop of its graph passes through the controller, and a visit to the controller either ends the run or makes a model call. A cycle that made no model call would leave $B-c$ unchanged, so the budget would not bound it. Such a cycle needs its own limit. For example, a loop that only reruns tools or checks would need a counter of its own.

The stall check does not provide a termination argument. It detects identical results. A run whose tools keep returning different but irrelevant results, for example searches with different patterns that each find unrelated lines, never triggers it. Such a run can continue without progress until the budget ends it. Detecting one failure pattern does not prove termination. In our agent, the termination argument rests on the call budget, provided that every step returns or times out.

Repetition has been observed in reactive agents. ReAct, short for reasoning and acting, is the pattern that the reactive loop of Lecture 2 follows. In a manual analysis of sampled ReAct trajectories on a question-answering task, one frequent error pattern was a model that generated its previous thoughts and actions again and failed to leave the loop [30, §3.3]. Our stall check detects one consequence of such repetition, identical observations.

The call budget, the stall check and the recursion limit can each end a run before the validator gives a verdict:

- **The call budget.** Each node that calls a model, namely `plan`, `controller` and `validate`, runs the admission check before it calls the model (Section 2). When `model_calls` has reached the budget of 40, the node returns the status `budget_exhausted` instead of making the call. The check runs before the call, because a check after the call would let the call incur its cost before the limit was enforced.
- **The stall check.** The first stall triggers plan revision (Section 3.3). A second stall, after the revision, ends the run with the status `no_progress`.
- **The recursion limit.** A **superstep** is one step of the graph runtime. The **recursion limit** caps the number of supersteps in a single run [9]. In our agent it is four times the call budget. It counts supersteps without regard to the task or to the cost of a step.

The budget and the recursion limit bound different quantities. The budget counts model calls, which carry the run's cost. The recursion limit counts supersteps of every kind, so it would also end a cycle that makes no model call. It would do so only after many supersteps, and it cannot distinguish a useful step from a wasted one. A cycle without model calls therefore still needs a limit of its own, tied to what the cycle does.

3.5 What a trace establishes

A trace can establish that a run terminated and that its acceptance procedure passed. No trace event can establish that the patch is correct. The following claims about a finished run must therefore be kept apart:

- **Execution stopped.** The **terminal event** is the event of kind `terminal`. It is normally the trace's last event, and its **terminal status** states how the run ended. A terminal event establishes that execution terminated.
- **Acceptance passed.** A terminal event with the status `accepted` establishes that the acceptance procedure passed. The patch passed the submit checks, and the validator accepted it.
- **The patch meets the postcondition.** In the specification of a method, the **postcondition** is the implementer's obligation, a condition on the program's state after the method runs [15]. For a task, it is the condition that the finished artifact, here the patch, must satisfy. No trace event establishes this claim.

For our patching task, the postcondition is what Lecture 2's performance measure asks: the weakness is gone, and the endpoint still behaves as it did. The acceptance procedure produces evidence about the patch. The submit checks show that the patch applies, touches the flagged line and leaves the rescan clean.

A clean rescan shows only that the scanner no longer recognizes the flagged pattern (Lecture 2), and none of the checks runs the endpoint. The validator adds a model's judgment of the patch. Passing this procedure therefore does not prove that the patch satisfies the postcondition. The trace records the outcome of the acceptance procedure. It does not establish that the patch is correct.

The model may propose that the work is finished, for example by replying without a tool call. The submit checks, the validator and the limits then decide how the run ends, and the code of the nodes sets the terminal status. The validator's verdict comes from a model, but the code of the `validate` node turns it into the status `accepted` or `rejected`. No model reply writes the status field directly.

A run of the planning agent ends with one of the following terminal statuses:

- `accepted`. The validator accepted a patch that had passed the submit checks.
- `rejected`. The validator rejected the patch.
- `budget_exhausted`. An admission check found the call budget spent before a model call.
- `no_progress`. The run stalled again after its one plan revision.
- `error`. An exception escaped the run.

The status `running` marks a run that is still active, so it is never a terminal status.

When a run ends normally, its trace must contain a terminal event. A run ends normally when it reaches `END` without an exception. Every exit of the graph sets the status field, and the runtime records that status in a terminal event before the trace is closed. The trace writer, the code that appends events to the trace and closes it, checks this rule at closure. If no terminal event has been written, closing the trace raises an exception that reports a run closed without stating how it ended.

Every normal ending sets a status, so a terminal event missing at normal closure means that the recording failed. Such a missing event is an **instrumentation defect**: a fault in the code that records the run, which did not record how the run ended.

An escaped exception and a killed process leave different records. If an exception escapes the run, the trace writer records a terminal event with the status `error`. It then writes a separate `state_change` event that names the exception, so in this case the terminal event is not the last event. A `SIGKILL` is the operating-system signal that ends a process immediately. The process cannot intercept it, so none of its code runs after the signal, and no terminal event is written.

Table 5 lists what a reader may conclude from each way a trace can end. The row that needs the most care is the last one, a trace with no terminal event.

Table 5: What a trace records about the end of a run, and what that record supports.

What the trace records	What it supports
A verdict, then a terminal event with status `accepted`	Execution terminated and the acceptance procedure passed. The trace does not establish that the patch meets the task's postcondition.
A terminal event with status `rejected`, `budget_exhausted`, `no_progress` or `error`	Execution terminated without acceptance, for the reason the status states. The events before the ending record how far the run got. For `error`, the next event names the exception.
No terminal event	The trace does not establish termination. The run may still be active, or the process may have been killed before it could write a terminal event.

Labeling a run `error` because its trace has no terminal event assigns a cause without evidence. The status `error` states that an exception escaped the run, and a trace without a terminal event does not show that. The run may still be active, or it may have been killed. The accurate reading is that the trace establishes neither whether the run ended nor how.

3.6 A recorded repair and a keyboard interruption

A second recorded run of the planning agent shows the repair edge taken twice and a run ended by an interruption. The run worked on a path-traversal alert in OWASP BenchmarkPython, an alert about a file path that may be built from outside input.

The first patch applied successfully. It left the flagged line unchanged, however, and the rescan still reported the alert. The trace shows the following sequence:

- **The checks.** The `submit` node recorded its check results: the patch applies, it does not touch the flagged line, and the rescan is not clean. It also recorded a reason for each failed check.
- **The routing event.** The next event routed the run to the controller with `by` set to `runtime`, because the tested value was a check result. It stated that the submit checks had failed and listed both reasons.
- **The repair.** The controller received both reasons as a conversation message. It then ran further rounds through the tools (Section 1) and submitted again.

The second submission left the rescan clean, but the patch still did not touch the flagged line. That check failed, the runtime returned its reason to the controller, and the repair edge was taken a second time.

After more rounds through the tools, a keyboard interrupt stopped the run. A keyboard interrupt is the signal sent when a user presses Ctrl-C, and Python raises it inside the program as the exception `KeyboardInterrupt`. The trace ends with the last tool result, a terminal event with the status `error`, and a `state_change` event that names the exception. This ending falls in the second row of Table 5. The trace establishes that execution terminated without acceptance, and it names the exception. It does not establish what the process was doing between the last tool result and the interrupt.

A `SIGKILL` at the same point would have left a different trace. The trace would end at the last tool result, with neither a terminal event nor the event that names an exception. It would fall in the last row of Table 5, and it would not establish that the run terminated.

Neither recorded run ended with the status `budget_exhausted`. What we have said about that ending is inferred from the transition rules. No recorded trace shows it.

Key points

- At each visit to a control location that has a routing function, the function tests the merged record and selects one destination, so one compiled graph can send a run along different paths.
- The `by` field of a routing event records whether the model or the runtime supplied the tested value, and the programmer writes every guard in both cases.
- For an active run, the routing function after the controller tests only whether the last reply contains tool calls, so text in a reply that claims the work is finished has no effect on routing.
- Every feedback edge needs a trigger, the information it passes back and a bound: revision is limited to one per run, repair by the call budget, and the proposed new attempt by an attempt counter that survives every reset and by the call budget.
- An acceptance predicate only defines success, so termination needs a loop variant, here the budget minus the calls made, which bounds only the cycles that make a model call, so a cycle without one needs a limit of its own besides the recursion limit, which counts supersteps without regard to the task, and the stall check, which detects one failure pattern, does not prove termination.
- A terminal event establishes that a run terminated and the status `accepted` establishes that the acceptance procedure passed, while no trace event establishes that the patch meets the task's postcondition.
- A trace without a terminal event does not establish termination, because the run may still be active or its process may have been killed, so labeling such a run `error` assigns a cause without evidence.

4 Vulnerability management and discovery (the HW1 domain)

The planning agent of the first three sections (Sections 1 to 3) receives one scanner alert and patches it without asking whether the alert points to a flaw that an attacker could use. The first homework, HW1, asks an agent to answer that question for every alert, and designing such an agent requires a precise statement of the task. This section places the task within vulnerability management, describes how discovery tools produce alerts, and introduces what is needed to judge an alert: the attack surface,

the threat model and the catalogues that classify flaws. Section 5 then turns to Bandit's alerts and to triage itself.

4.1 Vulnerability management and its stages

Vulnerability management is the work of finding the flaws in a program that an attacker could use, deciding which of them are real, proving that they are real, and fixing them. We use the plain word *flaw* for a defect of this kind. Section 5 defines the more precise term *vulnerability*.

The work has four stages, and each stage takes the result of the one before it:

- **Discovery.** Discovery turns a repository and a scope, the part of the code to be examined, into alerts. As in Lecture 2, an alert is a report that names a file, a line and the kind of flaw suspected there.
- **Triage.** Triage turns each alert into a label, together with the evidence for it. A **label** is the verdict on one alert. In HW1 it is 1 when an attacker, as the threat model defines one (Section 4.6), can reach the flaw the alert points to, and 0 when no such attacker can.
- **Validation.** Validation proves that a flaw kept by triage is real by reproducing it, for example by running the program on an input that makes the flaw occur.
- **Patching.** Patching changes the code so that a proven flaw is gone and the program still behaves as it did for legitimate use.

The stages narrow the work step by step. Discovery may report many alerts, triage keeps the ones an attacker can reach, validation confirms them, and patching removes them. A mistake in one stage carries into the next. An alert that triage wrongly keeps costs work later, and a real flaw that triage wrongly drops is never patched.

HW1 builds the first two stages. Its discovery stage is a fixed scan with one tool (Section 4.4), and the agent you build in HW1 performs triage. Validation and patching come later in the course, with HW2 and HW3.

The planning agent works at the last of these stages, patching. HW1 adds the triage stage that the planning agent skips.

Validation in this sense names a stage of vulnerability management. It is distinct from the planning agent's validator (Lecture 2, Section 4), which judges whether a submitted patch should be accepted.

4.2 Kinds of discovery tool

Tools discover flaws in different ways, and each way misses some flaws. A discovery tool reads the program's code, runs the program and watches how it behaves, or checks the libraries that the program depends on. Static application security testing (**SAST**) reads the code. Dynamic application security testing (**DAST**) and **fuzzing** run the program, and **dependency scanning** checks its libraries. Table 6 summarizes how each kind finds flaws, and the list after it gives what each kind misses.

Table 6: Kinds of discovery tool and how each one finds flaws.

Kind	How it finds flaws
SAST	Matches rules against the source code, without running the program
DAST	Runs the program and attacks it from outside, watching how it responds
Fuzzing	Feeds the program generated inputs and keeps the inputs that make it crash
Dependency scanning	Checks the program's libraries against published records of flaws

Each kind sees some flaws and misses others:

- **SAST.** A SAST tool reads the code without running it. Lecture 2 called a tool of this kind a static-analysis scanner. It can examine every line, including code that no test reaches. It never sees the values that arrive while the program runs, so where a value comes from must be inferred from the code. Bandit's rules make no such inference, because each one matches only the shape of the code (Section 5).
- **DAST.** A DAST tool runs the program, sends it requests from outside, as an attacker would, and watches its responses. It finds only flaws in the code that its requests cause to run.
- **Fuzzing.** Fuzzing is dynamic testing in which the inputs are generated automatically instead of crafted by hand. The tool keeps the inputs that make the program crash. Like DAST, it finds only flaws in the code that its inputs cause to run.
- **Dependency scanning.** Dependency scanning, also called software composition analysis, compares the libraries a program uses, and their versions, with published records of known flaws. It finds only flaws that are already known in those libraries. Flaws in the program's own code are outside its view.

HW1 discovers flaws with one SAST tool, Bandit, a static-analysis scanner for Python, which also produced the worked alert of Lecture 2 [22]. HW2 and HW3 add other kinds of discovery tool.

4.3 Rules and what they match

A SAST tool finds flaws with rules. A **rule** consists of a pattern, which is the shape of code to look for, a message to print when the pattern matches, and metadata that classifies the match.

Bandit's own rules are part of its Python source code (Section 5). Semgrep is another SAST tool, whose rules are written as short text files, so the parts of a rule can be read directly [24]. GitLab has rewritten Bandit's rules in Semgrep's format [7]. A rule rewritten for another tool is called a port.

A Semgrep pattern is a piece of code with gaps in it. A gap is written as a **metavariable**, a name that begins with `$` and contains only capital letters, digits and underscores, such as `$X`. A metavariable matches one element of code in its position, such as a variable, an expression or the name of a function. Three dots, called an ellipsis, match a sequence of zero or more items: the arguments of a call, statements, or the characters inside a string.

When a pattern matches, each metavariable is bound to the code that fills its gap, and the tool reports an alert at the matched line. Semgrep calls an alert a finding.

Listing 7 shows the port of Bandit's rule B610, with two of its three patterns and the end of its message left out.

Listing 7: Bandit's rule B610 as GitLab ported it to Semgrep, shortened.

```
- id: bandit.B610
  patterns:
  - pattern-either:
    - pattern: $X.objects.$FUNC(...).extra(...)
    # two longer patterns omitted
  message: This is a last resort.
       You should be careful when using QuerySet.extra due to SQLi ...
  metadata:
    cwe: "CWE-89: Improper Neutralization
       of Special Elements used in an SQL Command ('SQL Injection')"
    owasp: 'A1: Injection'
  severity: ERROR
  languages: [python]
```

The rule has the following parts:

- **Identifier.** The field `id` names the rule after the Bandit rule it ports.
- **Pattern.** The block `patterns` gives the shape of code to match, and `pattern-either` means that any one of the listed patterns may match. This pattern matches a call to `extra` at the end of a database query written with Django, a Python

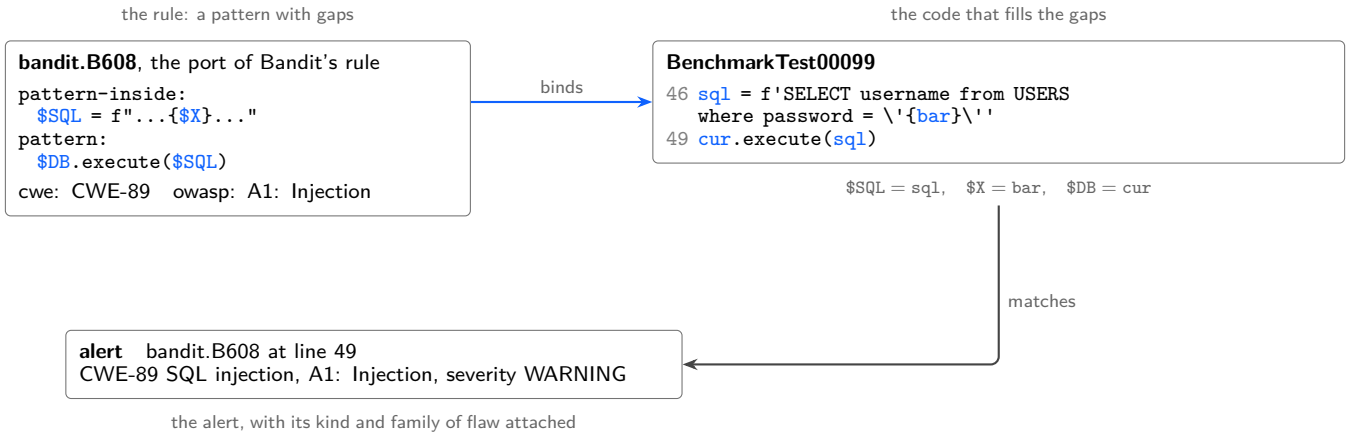


Figure 7: A Semgrep rule matching a BenchmarkPython test case. The metavariables of GitLab's port of Bandit's rule B608 bind to the variables of lines 46 and 49, and the match is reported as an alert that carries the rule's CWE number and Top 10 family.

web framework. In it, `$X` and `$FUNC` stand for any class and any method, and the dots stand for any arguments. The method `extra` lets a query include raw SQL text.

- **Message.** The field `message` is the text the tool prints with each match. Here it warns that `extra` can lead to SQL injection.
- **Metadata.** The field `cwe` gives the kind of flaw by its number in the Common Weakness Enumeration, a public catalogue of kinds of flaw. In this rule it is CWE-89, SQL injection. The field `owasp` names the flaw's family: the entry of the OWASP Top 10, a second public list, to which the flaw belongs. Sections 4.7 and 4.8 describe both lists.
- **Severity and language.** The field `severity` is the rule's estimate of the harm if a match is a real flaw. Semgrep grades it with words such as ERROR and WARNING, and Bandit grades it as LOW, MEDIUM or HIGH (Section 5). The field `languages` names the programming languages whose files the rule applies to.

A rule therefore carries the shape to match, the message to print, and the kind and family of flaw that a match belongs to.

Figure 7 shows a rule binding to real code. The rule is GitLab's port of Bandit's B608, which looks for SQL queries built from strings. Its `pattern-inside` clause requires that the match lie inside code where a variable `$SQL` is assigned an f-string with a gap `{$X}`. Inside the string, the dots match any characters, so the clause accepts any f-string with a gap. Its `pattern` clause then requires a call `$DB.execute($SQL)` with that variable as its only argument.

The code is the test case BenchmarkTest00099 of OWASP BenchmarkPython, the Python web application that the planning agent works on (Section 1). As in Lecture 2, a test case is a short program that handles one kind of web request. Line 46 builds a query as an f-string with `bar` in its gap, and line 49 executes it. The names set in blue are the metavariables and the code they bind to: `$SQL` = `sql`, `$X` = `bar` and `$DB` = `cur`. The port reports an alert at line 49 with CWE-89, the family tag A1: Injection, and the severity this rule assigns, WARNING.

A rule can hold several patterns, one for each way of writing the same flaw. Besides the form of Figure 7, the port's B608 entry matches a query built with the `%` operator, with `.format`, with an f-string or by concatenation directly inside the call to `execute`. It also matches a query built earlier with `%` or `.format` and stored in a variable.

Which code produces an alert depends only on the rule's pattern. GitLab's port writes the pattern of B608 in Semgrep's notation, while Bandit's own B608 is Python code that checks whether the text of a string looks like a SQL query (Section 5). The two patterns differ, so the two implementations report different alerts:

- **The reported line.** Bandit's own B608 reports the flaw of Figure 7 at line 46, where the query is built. The port reports line 49, where the query is executed.
- **The matched code.** Another test case, BenchmarkTest00011, writes its query as an f-string with no gap and passes the value to the database separately, as a bound parameter (Lecture 2). The port does not match it. Its query has no gap for `$X` to bind, and its call to `execute` passes a second argument, which `$DB.execute($SQL)` does not allow. Bandit's own B608 does report it, because it flags any f-string whose text looks like a SQL query, whether or not the f-string has a gap.

4.4 What an alert contains

In HW1, discovery runs Bandit's fixed set of rules over the code and reports every place where a rule matches. Each alert names a file, a line, the rule that matched and the kind of flaw that the rule suspects, so that the next stage knows where to look. An alert records only that a rule matched. Whether the match is a real flaw is left open, and deciding it is the job of triage.

The worked alert of Lecture 2 is an example. Bandit's rule B608 matched a line of BenchmarkPython that builds a SQL query by interpolation, and the alert named the file, that line and the rule. The alert did not say whether a value supplied by a user could reach the query. BenchmarkTest00099, the test case of Figure 7, is a different test case. It builds and runs its query with the same two lines as the worked alert's test case, at lines 46 and 49, and Bandit reports it in the same way, at line 46.

HW1 scans three targets with Bandit 1.9.4 and keeps the alerts that Bandit rates at MEDIUM severity or higher. The three targets are the following:

- **OWASP BenchmarkPython.** The first 99 test cases of BenchmarkPython give 18 alerts [20].
- **Radicale.** Radicale, a calendar server, gives 10 alerts [8].
- **SGLang.** SGLang, a server that runs large language models and answers requests to them, gives 21 alerts [27].

Each target is pinned at one commit, a fixed version of its code, and Bandit's rules are fixed as well. Scanning the same commit therefore gives the same list of alerts every time. Every agent built for HW1 receives the same alerts, and two runs of an agent can be compared alert by alert.

The targets play different roles in HW1. BenchmarkPython ships an **answer key**, a file that records the true labels: for each test

case, its category, the CWE number it tests, and whether its flaw is real. A **development set** is a target whose answer key is open to students, so that they can check their agent's labels while they build it. The first 99 test cases of BenchmarkPython are HW1's development set.

The agent itself never reads the answer key (Lecture 2, Section 2). An agent that could read it would copy its labels instead of analyzing the code. A **held-out target** is a target whose answer key stays hidden. Radicale and SGLang are held out, so they measure how the agent performs on code it was not tuned on.

4.5 The attack surface

An attacker can affect a program only through the data the program takes in from outside. That data arrives at a few places, such as a request, a file or a message, and an attacker starts there. The **attack surface** of a program is the set of places where data from outside the program enters it, together with the code that first handles that data.

> **Definition 4.1** (Attack surface)**.** "Intuitively, a system's attack surface is the set of ways in which an adversary can enter the system and potentially cause damage." [14, §1]

In the formal version of this definition, a system's attack surface consists of its entry and exit points, its channels and its untrusted data items [13]. For triage, the part that matters most is where data enters. An **entry point** is one place where data from outside enters the program.

Each HW1 target has its own attack surface:

- **BenchmarkPython.** Every test case is a Flask route, a function that the Flask web framework calls to handle requests to one address, and it reads a cookie or a form field from the request. Its surface is the request.
- **Radicale.** Radicale is a calendar server, so its surface is the HTTP request that a client sends, whether the client has logged in or not.
- **SGLang.** SGLang is a server for large language models. Its surface includes the request, and also the model files it loads and the channels over which its own worker processes exchange messages.

Listing 8 shows one entry point in code, the start of the test case of Figure 7.

Listing 8: The entry point of the test case BenchmarkTest00099 and the path of its value to `bar` (lines 31 to 42, with lines 34 to 37 and 40 left out).

```
31  param = request.form.get("BenchmarkTest00099")
32  if not param:
33      param = ""
38  conf52528 = configparser.ConfigParser()
39  conf52528.add_section('section52528')
41  conf52528.set('section52528', 'keyB-52528', param)
42  bar = conf52528.get('section52528', 'keyB-52528')
```

Line 31 is the entry point. It reads a form field of the incoming request, and the client who sends the request controls its value. Lines 32 and 33 replace a missing value with an empty string. Lines 38 to 42 store the value in a configuration object under one key and read it back under the same key, so `bar` holds exactly what the client sent. This detour is the benchmark's way of hiding a flow of data from a scanner.

A **sink** is the call where outside data can do harm, such as the call that sends a query to a database. In BenchmarkTest00099, line 46 places `bar` in the query text, and line 49 sends that text to the database (Figure 7). Line 49 is the sink, and whatever the client sends in the form field is the value of `bar` there. A scanner's rule usually points at the sink or at the line that prepares its argument. GitLab's port points at line 49, and Bandit's own B608 points at line 46 (Section 4.3).

For every alert, the agent you build in HW1 asks whether the value at the flagged line can come from the attack surface. In this test case it can. The value read at line 31 reaches the sink at line 49 unchanged.

4.6 The threat model

A label says whether an attacker can reach a flaw, so it needs a definite attacker: who the attacker is and which data they control. A **threat model** names the attacker, the data the attacker controls, and the harm the attacker must be unable to do. Table 7 gives the attacker of each HW1 target. Its last column, what the attacker controls, is what a label depends on.

Table 7: The attacker of each HW1 target and the data that attacker controls.

Target	The attacker	What they control
BenchmarkPython	An anonymous web client	Every byte of the request: cookies, form fields, headers
Radicale	A calendar client, logged in or not	The request, plus the storage directory if local users count
SGLang	A client, or whoever publishes a model	The request, the model files it loads, and its worker channel

For BenchmarkPython the threat model is simple, because an anonymous client controls the whole request. For Radicale and SGLang, the labels of most alerts depend on which attackers the threat model counts:

- **Radicale.** Seven of its ten alerts load files with Python's `pickle` module, which turns stored bytes back into Python objects and can run code chosen by whoever wrote the file. Radicale wrote these files itself, in its storage directory. The storage directory is in the attacker's control only if the threat model counts local users of the machine as attackers.
- **SGLang.** Eighteen of its 21 alerts load models, from model files or from the Hugging Face Hub, or unpickle bytes that do not come from the HTTP request: pickled files on disk, and messages on the channels over which SGLang's processes and their clients exchange work. Whether they are reachable depends on whether the threat model counts as an attacker whoever publishes a model, whoever writes those files, or whoever can send messages on those channels.

The labels of these alerts turn entirely on the threat model. Bandit ignores the threat model, because its rules match the shape of the code and carry no information about who controls the data.

A **rationale** is the text that justifies a label. The agent you build in HW1 should state the threat model once in its prompt, so that every label is decided under the same assumption. It should state it again in every rationale, so that a reviewer can see which attacker the label assumed. Without a stated threat model a label cannot be checked, because the same alert can deserve 1 under one threat model and 0 under another.

4.7 The OWASP Top 10

The **OWASP Top 10** is a ranked list of the most critical security risks to web applications, published by the OWASP Foundation (Open Worldwide Application Security Project). A **family** is one of its ten entries: a group of related kinds of flaw, such as injection. The current edition, the OWASP Top 10:2025, was announced as a release candidate on 6 November 2025 and published in final form in December 2025 [28]. Eight of its families were chosen and

Table 8: From a Top 10 family to a CWE, to the Python call that carries the flaw, and to the BenchmarkPython category that tests it. The CWE numbers are those of the benchmark's answer key.

Family	CWE	The Python call	Benchmark category
Injection	CWE-89	A query built from a string	`sqli`
Injection	CWE-78	A shell command built from a string	`cmdi`
Injection	CWE-94	`exec` or `eval` on a string	`codeinj`
Cryptographic Failures	CWE-328	MD5 or SHA1 applied to a secret	`hash`
Software or Data Integrity Failures	CWE-502	`pickle.load` or `torch.load`	`deserialization`
Security Misconfiguration	CWE-611	An XML parser applied to outside data	`xxe`
Broken Access Control	CWE-22	`open`, or a test that a file exists, on a path from outside	`pathtraver`

ranked from data on over 2.8 million applications, and two were added from a survey of practitioners. The analysis considered 589 kinds of flaw from the catalogue described in Section 4.8.

Table 9 lists the ten families with a plain description of each. The families that matter most for HW1 are Software or Data Integrity Failures, Injection, Cryptographic Failures and Security Misconfiguration. Table 8 follows these families down to the Python calls that carry their flaws, and Section 4.9 explains it.

Table 9: The ten families of the OWASP Top 10:2025, each with a plain description.

Family	What goes wrong
A01 Broken Access Control	A user reaches data or actions that should be denied to them
A02 Security Misconfiguration	A setting enables something that should be off
A03 Software Supply Chain Failures	A dependency brings in unchecked code
A04 Cryptographic Failures	Protection is missing or weak
A05 Injection	Outside data becomes part of a command
A06 Insecure Design	The design lacks a control it needed
A07 Authentication Failures	The program fails to confirm who the user is
A08 Software or Data Integrity Failures	Code or data is trusted without a check
A09 Security Logging and Alerting Failures	An attack is not recorded, or its record is not reviewed
A10 Mishandling of Exceptional Conditions	An error is handled in a way that grants access or reveals data

The edition matters when reading a rule's metadata. The tag `A1: Injection` of Listing 7 uses the numbering of the 2017 edition [21]. In the 2025 edition, Injection is A05.

The 49 alerts of HW1 fall into the families as follows:

- **Software or Data Integrity Failures.** This family has 26 alerts. They load pickled data or model files, or download models without an integrity check.
- **Injection.** This family has 7 alerts. They build SQL queries from strings, run `exec` or `eval`, or start a shell command.
- **Cryptographic Failures.** This family has 6 alerts, which use the MD5 or SHA1 hash.
- **Security Misconfiguration.** This family has 6 alerts, which parse XML. Section 4.9 explains why XML parsing belongs to this family.
- **None of the ten.** The remaining 4 alerts come from Bandit's rule B113, which no family covers (Section 4.9).

The integrity alerts of Radicale and SGLang are exactly the alerts whose labels turn on the threat model (Section 4.6). The other integrity alert comes from BenchmarkPython.

4.8 Weaknesses and their catalogues

A **weakness** is a kind of flaw that recurs across programs. Each family of the Top 10 is a set of weaknesses from a public catalogue. The Common Weakness Enumeration (**CWE**) is that catalogue, kept by MITRE, and it gives each weakness a number and a name [17]. An entry of the catalogue is also called a CWE. CWE-89, for example, is SQL injection, and it can occur in any program, in any language, that builds a query from text.

CWE, CVE and CVSS are easily confused, because they describe flaws at different levels:

- **CWE.** A CWE is a weakness, such as CWE-89. It describes no particular program.
- **CVE.** A Common Vulnerabilities and Exposures record (CVE) identifies one instance of a flaw in one product [2]. Its record usually names a CWE as the type of the flaw.
- **CVSS.** The Common Vulnerability Scoring System (CVSS) rates one instance of a flaw by how easily an attacker can use it and how much damage it does [5]. A CVSS score therefore attaches to a CVE.

A family groups many CWEs. In the 2025 edition, the Injection family alone contains 37 CWEs, including CWE-89 (SQL injection), CWE-78 (OS command injection), CWE-94 (code injection) and CWE-95 (eval injection) [28].

Every HW1 alert carries a CWE number, because Bandit 1.9.4 attaches one to every alert it reports. The number names the kind of flaw that the rule suspects at the flagged line.

4.9 From a family to a line of Python

The Top 10 and the CWE catalogue form a hierarchy that continues into the code: a family contains CWEs, and each CWE appears in code as a particular kind of Python call. Table 8 follows this hierarchy for several kinds of flaw that BenchmarkPython tests. Its last column names the benchmark category whose test cases cover each kind. The answer key has further categories, such as XPath injection (Section 5). Read each row from left to right, from the general family to the specific call.

The call at the bottom of the hierarchy is the sink (Section 4.5). An alert's CWE therefore usually tells the agent the family of the suspected flaw and the shape of code to look for around the flagged line.

The CWE numbers in Table 8 are those of BenchmarkPython's answer key. Bandit attaches its own numbers, which sometimes

differ from the key:

- **exec and eval.** Bandit reports CWE-78, OS command injection, where the key has CWE-94, code injection. Both are Injection CWEs.
- **MD5 and SHA1.** Bandit reports CWE-327, a broken or risky cryptographic algorithm, where the key has CWE-328, a weak hash. Both belong to Cryptographic Failures.
- **XML parsers.** Bandit reports CWE-20, improper input validation. The flaw these rules look for is an XML external entity (XXE) flaw, in which an XML document makes the parser read a file or contact a server that the document names. The key gives CWE-611 for its XXE cases. CWE-20 is an Injection CWE in the 2025 edition, while CWE-611 belongs to Security Misconfiguration.

The XML alerts are the exception to this correspondence. Their CWE points to Injection, while the flaw they look for belongs to Security Misconfiguration, which is where the counts of Section 4.7 place them. In the development set, the two XML alerts of BenchmarkTest00017 fall in a test case of the XXE category. The XML alerts of BenchmarkTest00013 to BenchmarkTest00016 fall in test cases of the XPath-injection category, for which the key gives CWE-643 (Section 5).

A scanner's rules and the families of the Top 10 are separate lists. A rule can report a flaw that no family contains, and a family can contain flaws that no rule reports. Bandit's rule B113 reports an HTTP request made without a timeout, which can leave the program waiting indefinitely, and it attaches CWE-400, uncontrolled resource consumption. CWE-400 belongs to none of the ten families, so the four B113 alerts of HW1 have no family.

Path traversal is a flaw of the second kind. In a path traversal (CWE-22, Broken Access Control), a request makes the program open a file outside the folder it intended. BenchmarkPython's path-traversal cases open a file, or test whether a file exists, on a path built from the request, and Bandit has no rule for either shape of code. The path-traversal cases of the development set therefore receive no Bandit alert (Section 5). The path-traversal alert of the second recorded run in Section 3 came from a different scanner.

Key points

- Vulnerability management has four stages: discovery turns code into alerts, triage turns each alert into a label with its evidence, validation proves a kept flaw by reproducing it, and patching removes a proven flaw while keeping the program's behavior.
- Discovery tools read the code (SAST), run the program (DAST and fuzzing) or check its libraries (dependency scanning), and a SAST tool such as Bandit matches rules, each made of a pattern, a message and metadata.
- An alert records only that a rule matched, and HW1's alerts come from one fixed Bandit scan of a development set with an open answer key (BenchmarkPython's first 99 test cases) and of two held-out targets (Radicale and SGLang).
- The attack surface is the set of places where outside data enters a program, with the code that first handles it, and triage asks whether the value at the sink can come from an entry point.
- A threat model names the attacker, the data the attacker controls and the harm to prevent, and because the same alert can deserve 1 under one threat model and 0 under another, the agent states it in its prompt and in every rationale.
- The OWASP Top 10:2025 ranks the most critical security risks to web applications as ten families, each a set of CWEs, where a CWE is a kind of flaw, a CVE is one instance of a flaw in one product, and a CVSS score rates that instance.
- An alert's CWE usually names its family and the shape of code to look for, although Bandit's XML alerts point to the wrong family and the B113 alerts belong to no family.

5 Bandit alerts and triage

Section 4 named the weaknesses that HW1's alerts point to and the attacker against whom each one matters. This section describes the alerts themselves, which are what the agent you build in HW1 receives and must label. We show how Bandit reads a program, what its rules and alert records contain, and when a matched weakness is a vulnerability, using alerts from the development set with and without a real flaw behind them. We then explain why every alert needs triage, what each kind of triage mistake costs, how alerts are prioritized, and what a label must carry.

5.1 How Bandit reads a program

Bandit finds alerts by matching shapes in the structure of a program, without running it. A **syntax tree** is the tree that a parser builds from source code. Each node stands for one construct of the language, such as an assignment, a function call or a name, and the children of a node are the parts of that construct. Bandit parses each Python file into the syntax tree that Python's own parser produces [23] and runs every rule against the nodes of the tree [22].

A rule, as Section 4 defined it, is a pattern, a message and metadata. In Bandit, the pattern is a shape in the syntax tree, for example a call to a function with a given name. When a node has that shape, the rule reports an alert at the line where the node occurs, together with the rule's message and its severity (Section 5.3).

What a rule can see is limited to the part of the tree it reads. A rule reads the node it matched and a few nodes near it, which record the call as it is written: which function is called and what is passed to it. The rule does not follow a name back through the assignments that gave the name its value. It therefore learns nothing about where an argument's value came from. When the argument is a variable, as `fb` is in Figure 8, the rule cannot tell whether the variable holds a value read from a request or a constant assigned a few lines earlier.

Every Bandit alert reports a matched shape and leaves open where the data at that shape comes from. Whether data from an attacker can reach the matched call is the question that triage answers (Section 5.10).

Figure 8 follows one match from a line of code to an alert. The line comes from Radicale, a calendar server and one of HW1's held-out targets. It loads a stored object from the open file `fb` and unpacks it into two variables. Python's parser turns the line into an assignment node. Its target is a tuple of two names, and its value is a call. The function called is the attribute `load` of the name `pickle`, and the one argument is the name `fb`.

Bandit reads the called name, `pickle.load`, along the highlighted path and looks it up in the list of names held by rule B301. The name is on the list, so B301 reports an alert on this line. The argument appears in the tree only as the leaf `Name fb`, and the rule never visits it. Even that leaf holds only the variable's name. The bytes that the file will contain exist only when the program runs, and whether an attacker could have written them decides whether this alert is real.

Bandit reads Python only. Other SAST tools, such as Semgrep and CodeQL, do the same job for many languages.

5.2 A rule in Bandit's source

In Bandit's source, the simplest rules are short data structures. Each holds a list of function names, a message, a CWE and a severity, and Bandit calls such a list of names a blacklist. Other rules, such as B102 for `exec` and B608 for SQL text, are short functions. Each of them inspects the matched node and returns a message, a CWE, a severity and a confidence.

Listing 9 shows the entry for B301, the rule that matched the line of Figure 8 [22]. The function `build_conf_dict` builds the

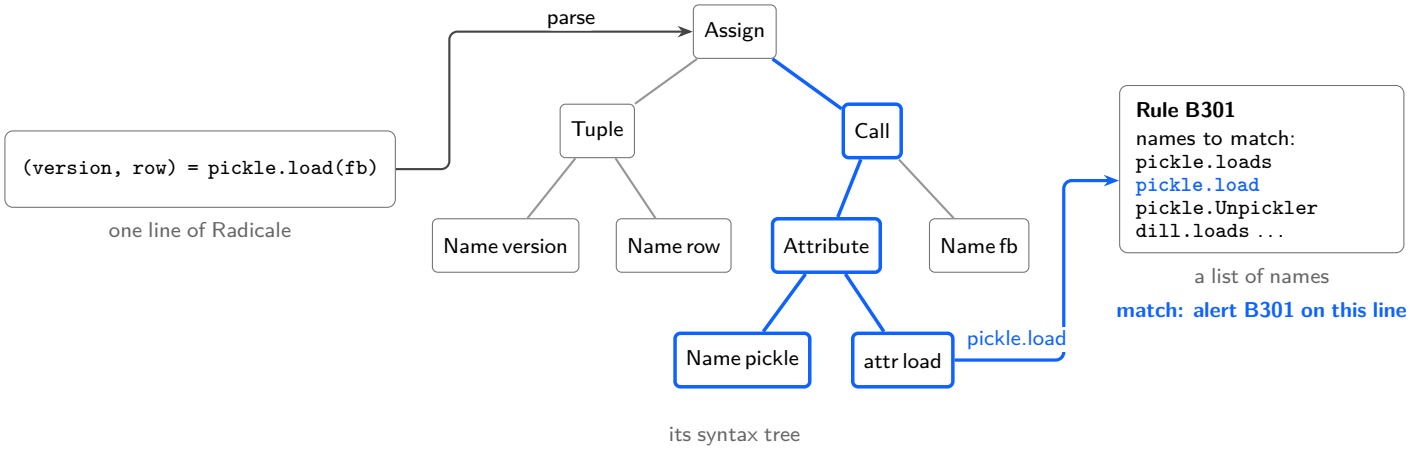


Figure 8: One line of Radicale, its syntax tree and Bandit's rule B301. The highlighted path leads from the assignment to the called name, `pickle.load`, which the rule finds in its list of names and reports as an alert. The argument `fb` is a leaf that the rule never reads.

entry from a name, an identifier, a CWE, the names to match, the message and an optional severity.

Listing 9: Bandit's entry for rule B301, with the list of names shortened.

```
utils.build_conf_dict(
    "pickle",
    "B301",
    issue.Cwe.DESERIALIZATION_OF_UNTRUSTED_DATA,
    [
        "pickle.loads",
        "pickle.load",
        "pickle.Unpickler",
        # ... nine more names, from dill.loads
        # to pandas.read_pickle
    ],
    "Pickle and modules that wrap it can be unsafe when used to "
    "deserialize untrusted data, possible security issue.",
)
```

The parts of the entry are the following:

- **Name and identifier.** The strings `"pickle"` and `"B301"` name the rule. The identifier B301 appears in every alert the rule produces.
- **CWE.** The constant `DESERIALIZATION_OF_UNTRUSTED_DATA` stands for CWE-502, deserialization of untrusted data. **Deserialization** is the step that turns stored bytes back into objects. Every alert of the rule carries this CWE.
- **Names to match.** The list holds the functions the rule matches, each written as a **qualified name**: the function's name together with the module it belongs to, as in `pickle.load`. Besides three names from `pickle`, the list holds nine names from modules that wrap it, from `dill` to `pandas.read_pickle`.
- **Message.** The two strings at the end form the text reported with each alert.
- **Severity.** The entry gives none, so its alerts take the default, MEDIUM.

The message states the reason for the rule. `pickle` is Python's format for saving objects as bytes. Loading a pickle can make Python call functions that the bytes name, so an attacker who controls the bytes can make the program run code of their choosing.

The rule knows nothing about what is unpickled. It matches the name of the function, so it reports every call to `pickle.load` in the same way, whether the bytes come from a file the program wrote itself or from a request. Every one of those alerts receives the same message, CWE and severity.

5.3 Severity and confidence

Every Bandit rule attaches two estimates to its alerts, a severity and a confidence. The **severity** of a rule is its estimate of the harm if the flaw is real. Its **confidence** is its estimate of how often the matched shape is that flaw. Bandit grades both as LOW, MEDIUM or HIGH.

Table 10 gives both estimates for the rules that fire on HW1's three targets, as Bandit 1.9.4, the version HW1 uses, reported them. Most of these rules estimate MEDIUM harm. The weak-hash rule B324 and the shell rule B602 estimate HIGH harm, and the rules for SQL text and for requests without a timeout have LOW confidence.

One rule adjusts its confidence to the code it matched. B608, the SQL rule, reports MEDIUM confidence when the SQL text is built inside the call that runs it. In the development set's cases the text is built in one statement and run in another, and there B608 reports LOW confidence.

B602, the shell rule, adjusts its severity to the code it matched in a similar way. It reports HIGH severity when the shell command is built at run time, as in Radicale's alert, and LOW severity when the command is a literal string. HW1's filter drops such LOW alerts.

Both estimates were made before anyone read the code around the match. They are decided when the rule is written, and the rule assigns them from the matched shape alone. B301's entry gives no confidence, and Bandit gives every alert of a rule built from a list of names HIGH confidence. Most rules give every alert the same two values, and even B608 looks only at the statement it matched.

Neither estimate says where the data at the matched call comes from. B301, for example, reports HIGH confidence on each of Radicale's pickle alerts, although whether any of them is a vulnerability depends on who can write the files they load (the threat model of Section 4).

HW1 filters alerts by severity alone. It keeps every alert of MEDIUM or HIGH severity, whatever its confidence. A rule with LOW confidence, such as B608 or B113, therefore still produces alerts that the agent must triage.

5.4 One run and one alert record

A Bandit run produces a list of alerts, and each alert reaches the agent as one structured record. HW1's brief fixes the command for each target. For Radicale, Bandit scans the server's package recursively, skips its tests, keeps the alerts of MEDIUM severity or higher and writes its results as JSON. At the commit HW1 uses [8], this run reports ten alerts:

- **B301 (pickle).** Seven alerts, each a call that loads pickled data, with MEDIUM severity and HIGH confidence.
- **B602 (shell).** One alert, a subprocess started through the shell, with HIGH severity and HIGH confidence.

Table 10: Severity and confidence of the Bandit rules that fire on HW1's three targets.

Rule	What it matches	Severity	Confidence
B608	SQL text built in a string	MEDIUM	LOW
B301, B614, B615	`pickle`, `torch.load`, downloads from the Hugging Face Hub not pinned to a commit	MEDIUM	HIGH
B102, B307	`exec`, `eval`	MEDIUM	HIGH
B314, B317, B318	Standard XML parsers, which are unsafe on outside data	MEDIUM	HIGH
B324	MD5 or SHA1 in `hashlib`	HIGH	HIGH
B602	A subprocess started through the shell	HIGH	HIGH
B113	Requests sent without a timeout	MEDIUM	LOW

- **B324 (weak hash).** One alert, a use of MD5 or SHA1, with HIGH severity and HIGH confidence.
- **B113 (timeout).** One alert, a request sent without a timeout, with MEDIUM severity and LOW confidence.

The same command on the same commit returns the same ten alerts, as Section 4 explained for every target. Any difference between the labels of two runs of a triage agent on Radicale therefore comes from triage.

Listing 10 shows the record of the alert on the line of Figure 8, shortened. The record is JSON, and its fields name the rule, give its estimates and its CWE, and locate the match:

- **Rule.** `test_id` names the rule, B301. `test_name` is `blacklist` for every rule built from a list of names. A rule written as a function puts the function's name there instead, such as `hashlib` for B324.
- **Estimates.** `issue_severity` and `issue_confidence` carry the rule's two estimates.
- **CWE.** `issue_cwe` gives the CWE number and a link to its entry in the catalogue. It names the kind of flaw the rule believes it found, here CWE-502.
- **Location.** `filename` and `line_number` say where the match is.

Listing 10: One Bandit alert record, as the agent receives it, with its message, code, position and documentation fields cut.

```
{
  "test_id": "B301",
  "test_name": "blacklist",
  "issue_severity": "MEDIUM",
  "issue_confidence": "HIGH",
  "issue_cwe": {
    "id": 502,
    "link": "https://cwe.mitre.org/data/definitions/502.html"
  },
  "filename": "radicale/sharing/files.py",
  "line_number": 131
}
```

The full record also holds the rule's message in `issue_text`, the exact position of the match in `line_range`, `col_offset` and `end_col_offset`, the lines around the match in `code`, and a link to the rule's documentation in `more_info`.

Nothing in the record says where the loaded bytes come from. Apart from the position and the code of the match, the record holds only what the rule supplies: its identifier, its message, its CWE and its two estimates. Finding out where the data comes from is left to the agent.

5.5 Vulnerability and exploit

A weakness becomes a vulnerability when a running program lets an attacker reach it. A weakness, as Section 4 defined it, is a kind of flaw, such as building a database query by pasting a value into its text. One occurrence of a weakness in a program counts as a vulnerability only when a threat source, such as an attacker, could exploit it.

Definition 5.1 (Vulnerability)**.** "Weakness in an information system, system security procedures, internal controls, or implementation that could be exploited or triggered by a threat source." [18]

Stated in terms of a program, a **vulnerability** is a weakness an attacker can reach in a running program. An **exploit** is an input that reaches it. A weakness in a program is a vulnerability when all of the following hold:

- **A path.** The attacker's data enters at the attack surface and travels to a sink, the call where it does harm (Section 4).
- **Nothing removes the danger.** No step on the way makes the data harmless. Binding the value as a parameter of a query, checking the input, and putting a constant in the value's place are examples of such steps.
- **The harm matters.** The result is a harm that the threat model says we must prevent, such as running code that the attacker wrote.

These conditions give a label its meaning. A label of 1 claims that a path from the attack surface to the sink exists under the threat model, with nothing on it that removes the danger. A label of 0 claims that no such path exists.

HW1 calls an alert with a label of 1 a potential vulnerability. The word *potential* marks that triage reaches its label by reading the code, without running it. Proving the flaw with a reproduction is the later stage of validation (Section 4).

A label of 1 makes a claim about a path, so its rationale has to show that path: where the data enters, how it travels and where it does harm. A rationale for a label of 0 names what blocks the path, such as a step that removes the danger. The lines of code that a reviewer can read to check a rationale are its **evidence**.

5.6 SQL injection: a real alert and a false positive

The same rule can fire on a real vulnerability and on safe code, because the shape it matches is present in both. An alert with no flaw behind it is a **false positive**, also called a false alarm. Listings 11 to 16 quote test cases of the development set, the first 99 test cases of OWASP BenchmarkPython [20], whose answer key says which cases are real. Each listing quotes the test case with its own line numbers, and a line `...` marks lines left out.

The first case is a real SQL injection, the kind of flaw that Lecture 2's worked alert patched. In Lecture 2, the query was built by interpolation: the code pasted a value into the query text. In Listing 11, the value of a form field that the client sends is stored in a configuration object and read back as `bar` at line 42, along the path that Section 4 traced from line 31.

Listing 11: A real SQL injection: BenchmarkTest00099, lines 42 to 49.

```
42  bar = conf52528.get('section52528', 'keyB-52528')
    ...
46  sql = f'SELECT username from USERS where password = \'{bar}\''
47  con = helpers.db_sqlite.get_connection()
48  cur = con.cursor()
49  cur.execute(sql)
```

Line 46 pastes `bar` into the query text with an f-string, and line 49 runs that text. The request's value becomes part of the statement, so it can change what the statement does. A password such as `' OR '1'='1`, for example, closes the quoted string early and adds a condition that every row satisfies. That input is an exploit. The label is 1, and the answer key marks the case as a real SQL injection, CWE-89. Bandit's B608 fires on the f-string.

The second case (Listing 12) draws an alert from the same rule, B608, because its query text is also written as an f-string. Line 41 sets `bar` to a constant, and the condition at line 42 is always true, so line 43 replaces it with `param`, the value of a cookie the client sends. This detour, like the one in Section 4, hides from a scanner that `bar` holds the client's value. The attacker's data therefore reaches the call at line 50.

Listing 12: A false positive from the same rule: BenchmarkTest00011, lines 41 to 50.

```
41  bar = "This should never happen"
42  if 'should' in bar:
43      bar = param
    ...
47  sql = f'SELECT username from USERS where password = ?'
48  con = helpers.db_sqlite.get_connection()
49  cur = con.cursor()
50  cur.execute(sql, (bar,))
```

The value travels as a parameter. As in the patch of Lecture 2, the query text holds a placeholder, the `?`, and the value is passed beside the statement as a bound parameter, the tuple `(bar,)`. The database treats a bound parameter as data only, so the value cannot change the statement. The f-string interpolates nothing, and B608 fires only because the string has the shape it matches.

The label is 0, and the placeholder and the tuple are its evidence. The answer key marks this case and its neighbor, BenchmarkTest00012, as safe. The attacker's data reaches the sink here, yet there is no vulnerability, because a step on the way removes the danger.

5.7 Code injection: `exec` and `eval`

A call that runs text as Python code is dangerous only when the text can hold code an attacker wrote. The function `exec` runs a string as Python statements, and `eval` evaluates a string as a Python expression. Bandit reports every call to either, B102 for `exec` and B307 for `eval`.

In Listing 13, `param` holds the value of a cookie, which the client can set to anything. Line 47 stores it in a configuration object, line 48 reads it back unchanged as `bar`, and line 51 passes it to `exec`.

Listing 13: A real code injection: BenchmarkTest00074, lines 47 to 51.

```
47  conf90091.set('section90091', 'keyB-90091', param)
48  bar = conf90091.get('section90091', 'keyB-90091')
    ...
50  try:
51      exec(bar)
```

Whatever the cookie holds runs as Python, so the label is 1. The answer key marks the case as a real code injection, CWE-94. Bandit's B102 attaches CWE-78, OS command injection, to the same call. The two numbers differ, and both belong to the Injection family (Section 4).

Listing 14 passes the same kind of cookie value to `eval`. Lines 41 to 43, left out of the listing, wrap the cookie value in two fixed strings, and the slice at line 44 cuts them off again, so `bar` is the cookie value.

Listing 14: A false positive on eval: BenchmarkTest00073, lines 44 to 54.

```
44  bar = string41416[4:-17]
    ...
46  if not bar.startswith('\'')
      or not bar.endswith('\'') or '\'' in bar[1:-1]:
        ...
50      return RESPONSE
    ...
52  try:
        ...
54          eval(bar)
```

The check at line 46 returns early unless `bar` starts and ends with a single quote and holds no other single quote. The only text that reaches `eval` is therefore a plain string literal, such as `'hello'`. Evaluating a string literal produces the string and runs no code.

The label is 0, and the check at line 46 is its evidence. The answer key marks the case as safe. B307 matched the call to `eval` as it matches every such call, and the check that removes the danger lies outside the shape it reads.

5.8 A weak hash

Some labels rest on the algorithm and its purpose, with no data flow to trace. In Listing 15, line 65 creates an MD5 hash object, line 66 feeds it the input, and line 68 takes the digest, the fixed-length value that the hash function computes from its input. Lines 69 and 70 append the digest to a password file. A later line of the case reports that a sensitive value was hashed and stored.

Listing 15: A weak hash for a sensitive value: BenchmarkTest00057, lines 65 to 70.

```
65  hash = hashlib.md5()
66  hash.update(input)
    ...
68  result = hash.digest()
69  f = open(f'{helpers.utils.TESTFILES_DIR}/passwordFile.txt', 'a')
70  f.write(f'hash_value={base64.b64encode(result)}\n')
```

Here the harm comes from the algorithm itself, so the call to `hashlib.md5` plays the role of the sink. MD5 is no longer considered strong enough to protect a sensitive value such as a password. The flaw is the choice of MD5 for this purpose, whatever value reaches it. A label of 1 on this alert therefore claims that a secret is protected with an algorithm too weak for it, and its rationale traces no path from the attack surface. The same call hashing a cache key would be label 0, because a cache key protects nothing secret.

The answer key marks all five hash cases that Bandit flags as real, with CWE-328, use of a weak hash. Bandit's B324 attaches CWE-327, use of a broken or risky cryptographic algorithm. Both belong to the Cryptographic Failures family.

5.9 An XML alert and an unreported XPath query

An alert can fall on a harmless line while a nearby line has the shape of a flaw that no rule reports. In Listing 16, B314 fires on line 55, a call to an XML parser. Parsing XML that an attacker supplies can be dangerous, which is why the rule exists. The file parsed here, however, is one that the program ships, so no attacker data reaches the parser. The alert is therefore a false positive, and its label is 0.

Listing 16: An XML alert on a fixed file and an XPath query: BenchmarkTest00013, lines 41 to 57.

```
41  bar = "alsosafe"
42  if param:
43      lst = []
44      lst.append('safe')
45      lst.append(param)
46      lst.append('moresafe')
47      lst.pop(0)
48      bar = lst[1]
    ...
54  try:
55      root = ET.parse(f'{helpers.utils.RES_DIR}/employees.xml')
56      query = f"/Employees/Employee[@emplid=\'{bar}\']"
57      nodes = elementpath.select(root, query)
```

Line 56 builds an XPath query by pasting `bar` into it. XPath is a query language that selects parts of an XML document, and pasting a request value into an XPath query is XPath injection, CWE-643. Bandit has no rule for this shape.

In this case the shape is harmless. Line 41 sets `bar` to the constant `'alsosafe'`. When `param` is not empty, lines 43 to 48 build the list `['safe', param, 'moresafe']`, remove its first element and take the second of those left, so `bar` becomes the constant `'moresafe'`. Either way, `bar` holds a constant at line 56.

The benchmark includes safe variants of each kind of flaw it tests, and this case is one of them. Its answer key lists it under XPath injection and marks it safe.

The real XPath cases of the development set show what discovery misses. A real XPath injection gets no alert at all, because Bandit has no rule for it, and the development set has two such cases. A real flaw that is left unreported, or reported and then dismissed, is a **false negative**. The two real XPath cases are false negatives of discovery.

5.10 What discovery leaves undecided

On the development set we can measure the scanner against the answer key, and the measurement shows what discovery leaves for triage. The 99 cases hold 34 real flaws. Bandit reports 18 alerts in 17 files, because one file carries two alerts. Each case is one file, so we count the alerts by file.

Table 11 sorts the outcomes. Its first two rows divide the 17 reported files, and its third row counts the real flaws that receive no alert. The last row picks out the path-traversal cases, whose 8 real flaws are among those 26.

Table 11: Bandit's alerts on the development set, compared with the answer key.

Outcome	Count
Reported, and real	8 files
Reported, and safe	9 files
Real, and unreported	26 cases
Path-traversal cases, no rule	21 cases, 0 alerts

The **precision** of a discovery tool is the share of its alerts that are real. Its **recall** is the share of the real flaws that it reports. These two measures summarize the table. Counted by file, Bandit's precision on the development set is 8/17, about 0.47, and its recall is 8/34, about 0.24.

About half of the alerts are false positives, and most of the real flaws go unreported. Of the 34 real flaws, 26 receive no alert. Some of them belong to kinds of flaw for which Bandit has no rule. The development set's 21 path-traversal cases (Section 4) receive no alert at all, and 8 of them are real.

Triage, the second stage of vulnerability management (Section 4), decides for each alert whether the match is a flaw an attacker can reach, using the code and the threat model. It works only on the alerts that discovery hands it. It can remove false positives and so raise precision. It cannot recover the real flaws that received no alert, so it cannot raise recall.

5.11 The cost of each kind of mistake

A triage stage can be wrong in two directions, and the two mistakes cost different people. Applied to triage, a false positive is a safe alert labeled 1 and passed on to a reviewer, and a false negative is a real flaw labeled 0 and dismissed. For a triage stage, precision is the share of the alerts labeled 1 that are real, and recall is the share of the real flaws among its alerts that it labels 1.

- **A false positive passed on.** It costs a reviewer's time, because someone must read the code to find that nothing is wrong. At scale it also costs the reviewers' trust in the tool, which is useful only while people act on what it reports.
- **A real flaw dismissed.** The flaw stays in the released product, and the cost falls on the users and systems that an attacker harms through it.

A 2026 study, available as a preprint, measured both mistakes for agents that filter the alerts of SAST tools [29]. It ran four tools, CodeQL, Joern, Semgrep and SonarQube, on the Java version of the OWASP Benchmark [19], a suite of test cases of the same kind as BenchmarkPython. Together the four tools flagged 98.3% of the benchmark's non-vulnerable cases.

Agents built on several agent frameworks then reviewed the alerts. The best configuration paired SWE-agent with the model Claude Sonnet 4. SWE-agent is the published reactive agent that Lecture 2 compared with Agentless. In this configuration, the share of non-vulnerable cases still flagged fell to 6.3%.

On the benchmark's real vulnerabilities, the same configuration labeled 22.25% as false positives, about a fifth. These are false negatives of triage, real flaws that the agent dismissed. Such agents are therefore suited to supporting a reviewer's decision. Letting them suppress alerts automatically and unconditionally would have discarded about a fifth of the real flaws on this benchmark.

The study shows, on one benchmark, that an agent can remove most false positives and that the same agent dismisses a substantial share of real flaws. It is a result on Java test cases. It does not measure Python code, Bandit, or the agent HW1 asks for.

The high share of false positives is not limited to benchmarks. In one industry report, about 91% of the alerts that SAST tools raised were noise, that is, false positives (Ghost Security, 2025, as quoted by Feiglin and Dar [4]).

On the development set, comparing the labels of the agent you build in HW1 with the answer key measures both kinds of mistake.

5.12 Prioritization

When more alerts are real than a team can fix at once, triage also orders them, using the signals it has. **Prioritization** is the ordering of alerts by expected harm, so that a reviewer's time goes to the worst plausible flaw first. The signals differ in what they cost to obtain:

- **Severity and confidence.** They come from the rule (Section 5.3), so they cost nothing to read. Most rules give the same pair to every alert, so these signals rarely separate two alerts of the same rule.
- **Harm in context and reachability.** The code shows how much harm a flaw of the alert's family would do in this program, and whether data from an entry point can reach the flagged line. An agent learns both by reading that code, which costs model calls.
- **Signals about exploitation.** Public sources describe how published vulnerabilities are exploited. The Known Exploited

Vulnerabilities catalogue of the US Cybersecurity and Infrastructure Security Agency (CISA) lists flaws that have been exploited in the wild [3]. The Exploit Prediction Scoring System (EPSS) estimates the probability that a published vulnerability will be exploited in the wild in the next 30 days [6]. Both describe published vulnerabilities that have a CVE identifier (Section 4). HW1 leaves both out.

HW1's label is the coarsest priority a triage stage can give. It sorts the alerts into two groups, the potential vulnerabilities that deserve a reviewer's time (label 1) and the rest (label 0), with no order inside either group.

5.13 What a label must carry

A label is useful only when a reviewer can check it, so each label carries a rationale and evidence. The rationale, as Section 4 defined it, is the text that justifies the label. HW1 records the evidence as a list of line ranges in the target's files. Listing 17 shows one row of HW1's output file, `report.json`, in the format of the brief, filled in for the eval case of Section 5.7.

Listing 17: One row of report.json for the alert on eval in BenchmarkTest00073, with three fields cut.

```
{
  "alert_id": "owasp-BenchmarkTest00073-py-54-B307",
  "test_id": "B307",
  "line": 54,
  "label": 0,
  "status": "classified",
  "rationale":
      "Attacker: an anonymous client who controls the request.
      Line 46 admits only a quoted literal, so eval runs no code.",
  "evidence": [
    {"path": "testcode/BenchmarkTest00073.py",
     "start_line": 44, "end_line": 54}
  ]
}
```

The fields of the row are the following:

- **Identity.** `alert_id` names the alert by target, test case, line and rule. `test_id` and `line` repeat the rule and the line. The full row also carries the alert's path, severity and confidence.
- **Label.** The label is 0, for the reason given in Section 5.7.
- **Status.** The row's status says whether the agent classified the alert. It is `classified` when the agent did, `budget_exhausted` when the run's token budget ran out before the agent classified the alert, and `error` when classification failed. HW1's token budget caps the tokens that all model calls of one run may use, across every alert. It differs from the planning agent's call budget, which counts model calls. The row's status is the status of this row and differs from the terminal status of a run (Section 3).
- **Rationale.** Two sentences state the threat model and the reason: the check at line 46 admits only a quoted literal, so `eval` runs no code.
- **Evidence.** Lines 44 to 54 of the test case hold the line that sets `bar`, the check and the call to `eval`.

A reviewer accepts a label only when the rationale and the evidence let them check it. With this row, a reviewer reads the cited lines and can confirm the label without repeating the agent's investigation. A label of 1 needs the same support. Its rationale names the entry point, the path and the sink (Section 5.5), or, for a weak hash, the secret that the algorithm protects (Section 5.8), and its evidence covers each of them.

Every rationale also states the threat model it assumed (Section 4). For BenchmarkPython that is an anonymous web client who controls the whole request, as the rationale of Listing 17 states.

Key points

- Bandit parses each Python file into a syntax tree and reports an alert wherever a rule's shape matches, such as a call to a named function, so an alert records a matched shape and nothing about where the argument's value came from.
- The simplest Bandit rules, such as B301, are lists of qualified function names with a CWE, a message and a severity, and every rule assigns its severity and confidence from the matched shape alone, so neither estimate says whether an attacker can reach a particular match.
- A vulnerability is a weakness an attacker can reach in a running program, and a label of 1 claims, under the threat model, a path from the attack surface to the sink with nothing on it that removes the danger, or, for a weak hash, a secret protected by too weak an algorithm.
- The same rule fires on real vulnerabilities and on false positives, because a bound parameter, a check on the input or a constant value removes the danger without changing the shape the rule matches.
- On the development set, about half of Bandit's alerts are false positives (precision about 0.47 by file) and most real flaws receive no alert (recall about 0.24), and triage can raise the first figure but never the second.
- A false positive passed on by triage costs reviewers time and trust, a dismissed real flaw stays in the released product, and in one study agents removed most false positives while dismissing about a fifth of the real vulnerabilities.
- HW1's label of 1 or 0 is the coarsest form of prioritization, the ordering of alerts by expected harm, and a reviewer accepts it only when its rationale and line evidence let them check it.

References

[1] Cornell CS 2112. Loop invariants. Lecture notes, CS 2112, Cornell University, https://www.cs.cornell.edu/courses/cs2112/2022fa/lectures/loopinv/, 2022. Fall 2022. Accessed 2026-09-24.

[2] CVE Program. Common vulnerabilities and exposures (CVE). https://www.cve.org/, 2026. The program identifies, defines and catalogs publicly disclosed cybersecurity vulnerabilities, with one CVE Record per vulnerability; operated by MITRE and sponsored by CISA. Accessed 2026-10-15.

[3] Cybersecurity and Infrastructure Security Agency. Known exploited vulnerabilities catalog. https://www.cisa.gov/known-exploited-vulnerabilities-catalog, 2026. Describes itself as the authoritative source of vulnerabilities that have been exploited in the wild. Accessed 2026-10-15.

[4] Jake Feiglin and Guy Dar. SastBench: A benchmark for testing agentic SAST triage, 2026. Preprint. arXiv:2601.02941v1, 6 January 2026. The figure of 91% noise among 2,166 flagged vulnerabilities (Section 2.1) is quoted from an industry report by Ghost Security (2025), not measured by the authors.

[5] FIRST. Common vulnerability scoring system version 4.0: Specification document. Forum of Incident Response and Security Teams, https://www.first.org/cvss/v4-0/specification-document, 2023. First published 1 November 2023; document revision 1.2, 18 June 2024. Base metrics: exploitability and impact. Accessed 2026-10-15.

[6] FIRST. Exploit prediction scoring system (EPSS). Forum of Incident Response and Security Teams, https://www.first.org/epss/, 2026. A model that estimates the probability that a published CVE will be exploited in the wild in the next 30 days. Accessed 2026-10-15.

[7] GitLab. Semgrep analyzer: rules/bandit.yml. https://gitlab.com/gitlab-org/security-products/analyzers/semgrep/-/blob/e0f35024/rules/bandit.yml, 2026. GitLab's port of Bandit's rules to Semgrep, at commit e0f35024: bandit.B608 at line 1179, bandit.B610 at line 1231, each tagged with a CWE and the 2017 Top 10 category "A1: Injection". Accessed 2026-10-15.

[8] Kozea. Radicale: A simple CalDAV (calendar) and CardDAV (contact) server. https://github.com/Kozea/Radicale, 2026. HW1 held-out target, pinned at commit eff8027f3dc4910be1659ee71f4b4a454ade5a8c. Accessed 2026-10-15.

[9] LangChain. LangGraph graph API. https://docs.langchain.com/oss/python/langgraph/graph-api, 2026. HW1 setup material, not assigned reading.

[10] LangChain. Persistence. https://docs.langchain.com/oss/python/langgraph/persistence, 2026. Accessed 2026-09-18. HW1 reference material, not assigned reading.

[11] LangChain. LangGraph runtime. https://docs.langchain.com/oss/python/langgraph/pregel, 2026. Accessed 2026-09-24. HW1 reference material, not assigned reading.

[12] Jukka Lehtosalo. PEP 589: TypedDict, type hints for dictionaries with a fixed set of keys. https://peps.python.org/pep-0589/, 2019. Accessed 2026-09-18. Final; incorporated into the typing module.

[13] Pratyusa K. Manadhata and Jeannette M. Wing. An attack surface metric. *IEEE Transactions on Software Engineering*, 37 (3):371–386, 2011. doi: 10.1109/TSE.2010.60. Intuitive definition: "the set of ways in which an adversary can enter the system and potentially cause damage"; formally the entry and exit points, the channels and the untrusted data items of a system.

[14] Pratyusa K. Manadhata, Kymie M. C. Tan, Roy A. Maxion, and Jeannette M. Wing. An approach to measuring a system's attack surface. Technical Report CMU-CS-07-146, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, August 2007. URL https://www.cs.columbia.edu/~wing/publications/CMU-CS-07-146.pdf. Supersedes CMU-CS-05-155, "An Attack Surface Metric".

[15] MIT 6.031. Reading 6: Specifications. Course reading, 6.031 Software Construction, Massachusetts Institute of Technology, https://web.mit.edu/6.031/www/sp21/classes/06-specifications/, 2021. Spring 2021. Accessed 2026-09-24.

[16] MIT 6.042. Lecture 16: State machines and invariants. Lecture notes, 6.042 Mathematics for Computer Science, Massachusetts Institute of Technology, https://courses.csail.mit.edu/6.042/past-devel/archive/spring03/handouts/lectures/lec16.pdf, 2003. April 10, 2003. Accessed 2026-09-24.

[17] MITRE. CWE: Common weakness enumeration. https://cwe.mitre.org/, 2026. A community-developed list of software and hardware weakness types, managed by MITRE and sponsored by CISA. Accessed 2026-10-15.

[18] NIST Computer Security Resource Center. Vulnerability. Glossary, https://csrc.nist.gov/glossary/term/vulnerability, 2026. Definition: "Weakness in an information system, system security procedures, internal controls, or implementation that could be exploited or triggered by a threat source." Sources listed include FIPS 200 and NIST SP 800-53 Rev. 5. Accessed 2026-10-15.

[19] OWASP Benchmark Project. OWASP benchmark. https://owasp.org/www-project-benchmark/, 2026. A Java and Python test suite designed to verify the speed and accuracy of vulnerability detection tools. The Java benchmark, version 1.2 (2,740 test cases), is the one Xiong and Zhang evaluate on. Accessed 2026-10-15.

[20] OWASP Benchmark Project. BenchmarkPython. https://github.com/OWASP-Benchmark/BenchmarkPython, 2026. Pinned at commit f1291485808b66e20ddb6b01b10dc71b3df8c8ba (2,540 files, 1,230 test cases; verified 2026-09-15). Answer keys and bundled scan results are stripped from the copy the agent sees.

[21] OWASP Foundation. OWASP Top ten 2017. https://owasp.org/www-project-top-ten/2017/, 2017. The 2017 edition, whose first category is A1:2017-Injection. Accessed 2026-10-15.

[22] PyCQA. Bandit: Security oriented static analyser for Python code. Documentation https://bandit.readthedocs.io/; source https://github.com/PyCQA/bandit, 2026. Version 1.9.4 (released 25 February 2026), the version HW1 pins. "Bandit processes each file, builds an AST from it, and runs appropriate plugins against the AST nodes." Accessed 2026-10-15.

[23] Python Software Foundation. ast — abstract syntax trees. Python 3.12 documentation, https://docs.python.org/3.12/library/ast.html, 2026. Accessed 2026-10-15.

[24] Semgrep. Rule structure syntax. Semgrep documentation, https://docs.semgrep.dev/writing-rules/rule-syntax, 2026. Required fields id, message, severity, languages and a pattern; metavariables begin with $. Accessed 2026-10-15.

[25] Sanjit A. Seshia. Discrete dynamics: State machines. Lecture slides, EECS 149/249A Introduction to Embedded Systems, UC Berkeley, https://ptolemy.berkeley.edu/projects/chess/eecs149/lectures/DiscreteDynamics.pdf, 2015. Slides © 2008–2015 E. A. Lee, A. L. Sangiovanni-Vincentelli, S. A. Seshia. Accessed 2026-09-24.

[26] Sanjit A. Seshia. Extended finite state machines, timed automata, hybrid automata. Lecture slides, EECS 149/249A Introduction to Embedded Systems, UC Berkeley, https://ptolemy.berkeley.edu/projects/chess/eecs149/lectures/ExtendedAndTimedAutomata.pdf, 2015. Slides © 2008–2015 E. A. Lee, A. L. Sangiovanni-Vincentelli, S. A. Seshia. Accessed 2026-09-24.

[27] SGLang Project. SGLang: A high-performance serving framework for large language models and multimodal models. https://github.com/sgl-project/sglang, 2026. HW1 held-out target, pinned at commit bbe9c7eeb520b0a67e92d133dfc137a3688dc7f2. Accessed 2026-10-15.

[28] Andrew van der Stock, Brian Glas, Neil Smithline, Tanya Janca, and Torsten Gigler. OWASP Top 10:2025. OWASP Foundation, https://top10.owasp.org/2025/, 2025. Announced 6 November 2025 at OWASP Global AppSec. Eight categories chosen from data contributed on over 2.8 million applications and two from a community survey; 589 CWEs analyzed. Category names as on the site: A01 Broken Access Control to A10 Mishandling of Exceptional Conditions. Accessed 2026-10-15.

[29] Yunpeng Xiong and Ting Zhang. Sifting the noise: A comparative study of LLM agents in vulnerability false positive filtering, 2026. Preprint. arXiv:2601.22952v3, 23 July 2026 (v1, 30 January 2026). The numbers used in these notes appear unchanged in v1 and v3: on the OWASP Benchmark for Java (version 1.2), at least one scanner flags 1,303 of 1,325 non-vulnerable test cases, a 98.3% baseline false positive rate (Section 4.1.2); SWE-agent with Claude Sonnet 4 reduces it to 6.3% (Section 4.1.2); that configuration labels 314 true vulnerabilities as false positives, a miss rate of 22.25% (Section 5.2); Lesson 3 (Section 5.5) advises decision support rather than unconditional automatic suppression. The four scanners are CodeQL, Joern, Semgrep and SonarQube (Table 2). The abstract's "over 92%" is the relative reduction (92.1%); the body gives the baseline as 98.3%. The v3 PDF carries the header of Proc. ACM Softw. Eng. 3(ISSTA), Article ISSTA009, publication date October 2026.

[30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. ICLR version, arXiv:2210.03629v3.